

UBL128 Serving: Prefill/Decode 解耦推理系统设计（带 Prefix 支持）

本文档描述一个 prefill/decode 解耦的推理服务系统的完整设计，并支持 prefix caching。

1. 硬件系统

1.1 UBL128 HBD（UB High Bandwidth Domain）

UBL128 是一个 **UB High Bandwidth Domain (HBD)**，是本系统中最小的紧耦合计算单元，规格如下：

- **构成**：8 台 PC16 服务器
- **CPU 总数**： $8 \times 2 = 16$ 颗 CPU
- **NPU 总数**： $8 \times 16 = 128$ 颗 **Ascend 950** 芯片
- **域内互联**：采用 **单层 Scale Up 交换机**，为每颗 NPU 提供 **3.2 Tbps** 的交换带宽
- 域内 128 颗 NPU 之间构成 full any-to-any 的高带宽互联，组成 **Scale Up (SU) 网络**

1.2 PC16 服务器

每台 PC16 服务器内部由 2 颗 CPU 和 16 颗 Ascend 950 NPU 组成，对外提供 3 类网络接口。

服务器内部 **NPU 拓扑**：

- 16 颗 NPU 分成 **两组，每组 8 颗（ 2×8 NPU）**，每组与 1 颗 CPU 形成一个 NUMA 单元
- 同一组内的 8 颗 NPU 在 **服务器内部** 采用 **400 Gbps 全互联（full mesh）**
- 每颗 CPU 通过 **8×400 Gbps UB**（合计 3.2 Tbps）分别连接到本组内的 **8 颗 NPU**
 - CPU 0 $\leftrightarrow$ NPU 0..7（一对一 400 Gbps UB）
 - CPU 1 $\leftrightarrow$ NPU 8..15（一对一 400 Gbps UB）
 - 用于 CPU 主存 $\leftrightarrow$ NPU HBM 的高带宽数据搬运（如 KV swap、prefix load、参数加载等）
- 跨组的 NPU 通信走 **服务器外部** 的 SU 交换机

服务器对外接口：

接口	带宽	连接的网络	用途
每颗 NPU $\times 1$	3.2 Tbps	Scale Up (SU) 单层交换机	构成 UBL128 域内 128 NPU 间 any-to-any 高带宽网络
每颗 NPU $\times 1$	400 Gbps	Scale Out (SO) 网络（UBG 协议）	跨 UBL128 的 NPU $\leftrightarrow$ NPU、NPU $\leftrightarrow$ CPU 通信
每颗 CPU $\times 8$	8×400 Gbps UB	服务器内部 $\rightarrow$ 同 NUMA 的 8 颗 NPU	CPU $\leftrightarrow$ NPU 高带宽数据通路（KV swap / 加载 / DMA）
每台服务器：CPU 网卡 1	400 Gbps	RoCE 数据中心网络（DCN）	通用以太网通信、控制面、外部服务接入
每台服务器：CPU 网卡 2	400 Gbps	Scale Out (SO) 网络	CPU 接入 SO 网络

1.3 三类网络总览

系统中共存在 三类物理网络，定位互不重叠：

1. Scale Up (SU) 网络 —— UBL128 域内

- 单层无阻塞交换机
- 128 颗 NPU，每颗 3.2 Tbps
- 用于 HBD 内部张量并行 / KV 高带宽传输 / 紧耦合集合通信

2. Scale Out (SO) 网络 —— 跨 UBL128

- 物理协议：**UBG** (UB 协议族的 SO 形态)
- 拓扑：**两层无收敛 CLOS**
- 端口规模：可提供 **16384** 个交换端口
- 接入对象：每个 UBL128 中的 **每颗 CPU 和每颗 NPU** 都有一条 400 Gbps UBG 接口
- 能力：实现多个 UBL128 HBD 之间，CPU 与 NPU 的 **any-to-any** 互联互通
- 承载流量：
 - **数据面**：跨 HBD 的 KV cache 迁移、prefill→decode 的 KV 推送、prefix 共享、跨域并行
 - **热路径控制面**：与计算 / KV 紧耦合的内部 RPC ($F \leftrightarrow M$ 、 $F \leftrightarrow PC/DC$ 、 $PC/DC \leftrightarrow M$) 走 **uRPC over UB Urma** (见下条) —— μs 级 RTT，与计算粒度匹配
- **UB Urma / uRPC 协议栈** (用于 SO/SU 上的内部 RPC)：
 - **UB** (Unified Bus)：华为统一总线协议族，覆盖 SU / SO / 服务器内部 NPU↔CPU 三段
 - **Urma** (UB Reliable Memory Access)：UB 上的可靠内存访问语义层 (类似 RDMA)，提供零拷贝、远端内存读 / 写、原子原语
 - **uRPC**：基于 Urma 的 RPC 协议；走 UB 网络专用，**RTT 在 μs 级**、零拷贝、与数据面共栈 (不需要 TCP/IP)

3. RoCE 数据中心网络 (DCN)

- **所有 CPU 接入 DCN**：每颗 PC16 内的 CPU、每台鲲鹏 CPU 服务器中的 CPU 都在 DCN 上可达；**NPU 不接入 DCN**
- 物理接入：每台 PC16 服务器一条 400 Gbps NIC (CPU 0 / CPU 1 共享)；每台鲲鹏 CPU 服务器经 1825 网卡的 1 × 400 Gbps 上行
- 用途：
 - **外部接入**：对外 **HTTP/gRPC ingress** (外部 user 用 IP 协议族)；外部监控 / 日志收集出口
 - **运维与管理面**：集群部署 / 配置下发 / Prometheus 指标拉取 / 健康检查 / SSH 等 —— 这些非 **hot-path** 的内部服务调用仍然走 **DCN**，使用 **gRPC 或 JSON/HTTP** (与既有运维生态兼容)
 - **基础设施数据面 (非 KV)**：DCN 上预先部署的 **POSIX-compliant 分布式文件系统** (NFS / 3FS / 类 Amazon S3 对象存储)，承担 **KV cache 以外的所有持久化数据访问** (含 §3.6.7 KV Meta server 的 snapshot 落盘)；以及 **TCP/IP 接入网**，承担与外部 IP 世界的通用对接 (镜像、外部数据集等)。详见 §1.7

协议-网络映射规则 (v1 硬约束)：

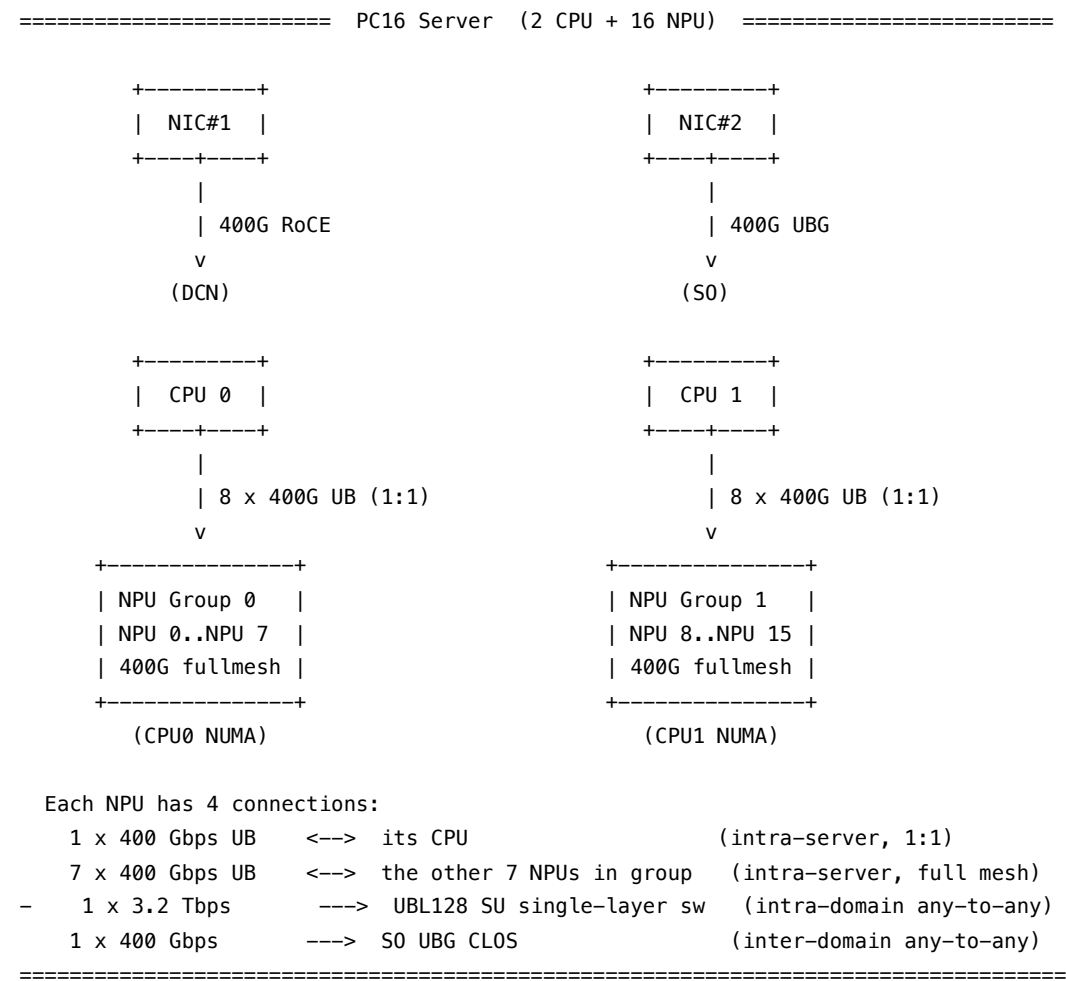
- 走 **DCN/RoCE** 的 **RPC** 一律使用 **gRPC (或 JSON/HTTP)** (与外界生态兼容、运维标准化)
- 走 **SO/SU (即 UB 网络)** 的 **RPC** 一律使用 **uRPC over UB Urma**，不使用 **gRPC** (gRPC 不为 UB 设计；HTTP/2 over UB 既低效也无意义)
- 因此：选择哪条网络 = 选择哪种协议；具体某个 RPC 走哪条由“是否 hot-path”决定 (hot-path → SO uRPC；非 hot-path → DCN gRPC 都可行)

1.4 拓扑示意 (自底向上 4 层视图)

说明：以下 4 张 ASCII 图中，所有方框 (+---+ | | +---+) 内部只放 **ASCII 字符**，方框中心列严格对齐；所有中文 (标题、连线标注、说明) 一律放在方框之外，避免 CJK 双宽度字符破坏对齐。

1.4.1 PC16 服务器（最小算力机柜单元）

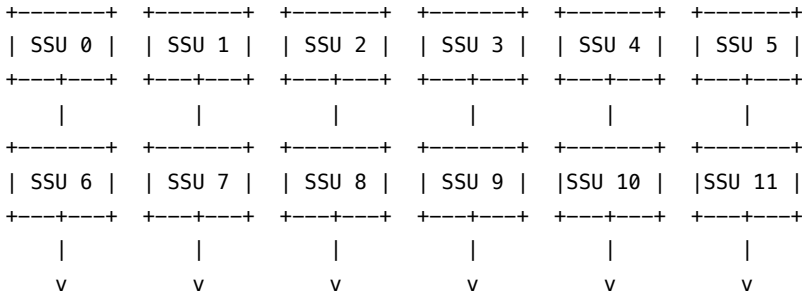
关键：每颗 CPU 通过 8 × 400 Gbps UB 链路一对一直连本组的 8 颗 NPU（CPU0↔NPU0..7, CPU1↔NPU8..15），构成两个 CPU-NPU NUMA 域。



1.4.2 SSU12 存储框（持久化 KV / Prefix / 模型权重的存储侧机柜）

SSU = 存储服务单元；SSU12 = 一个机框内含 12 个 SSU。每个 SSU 各自独立 直出 1 条 400 Gbps UBG 上行至 SO 网络（不经过任何聚合层）。

===== SSU12 Storage Rack (1 frame = 12 SSU) =====



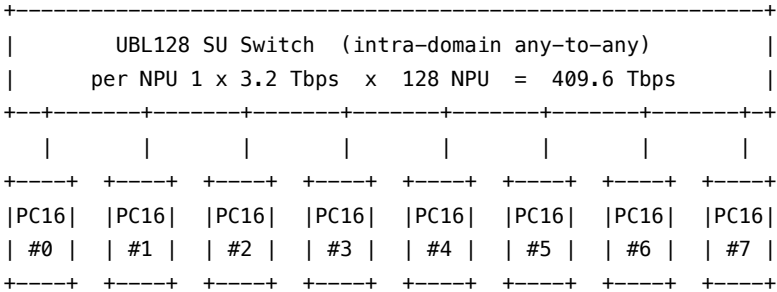
===== 12 x 400 Gbps UBG (each SSU 1 link, no aggregation) =====> S0 Network

- * Each SSU directly outputs 1 x 400G UBG -> S0; no in-frame aggregation
- * One SSU12 frame = 12 x 400G external uplinks
- * Purpose: persistent KV / Prefix / model weights

=====

1.4.3 UBL128 HBD (高带宽计算域)

===== UBL128 HBD (1 HBD = 8 PC16 = 16 CPU + 128 NPU = 128 Ascend 950) =====



- UBL128 external uplinks:
- 128 x NPU : each 1 x 400 Gbps UBG --> S0 (inter-domain data plane)
 - 16 x CPU : each PC16 1 x 400G RoCE --> DCN (control plane)
 - each PC16 1 x 400G UBG --> S0 (CPU data plane)
 - 128 x NPU intra: 1 x 3.2 Tbps -> SU switch (stays inside HBD)

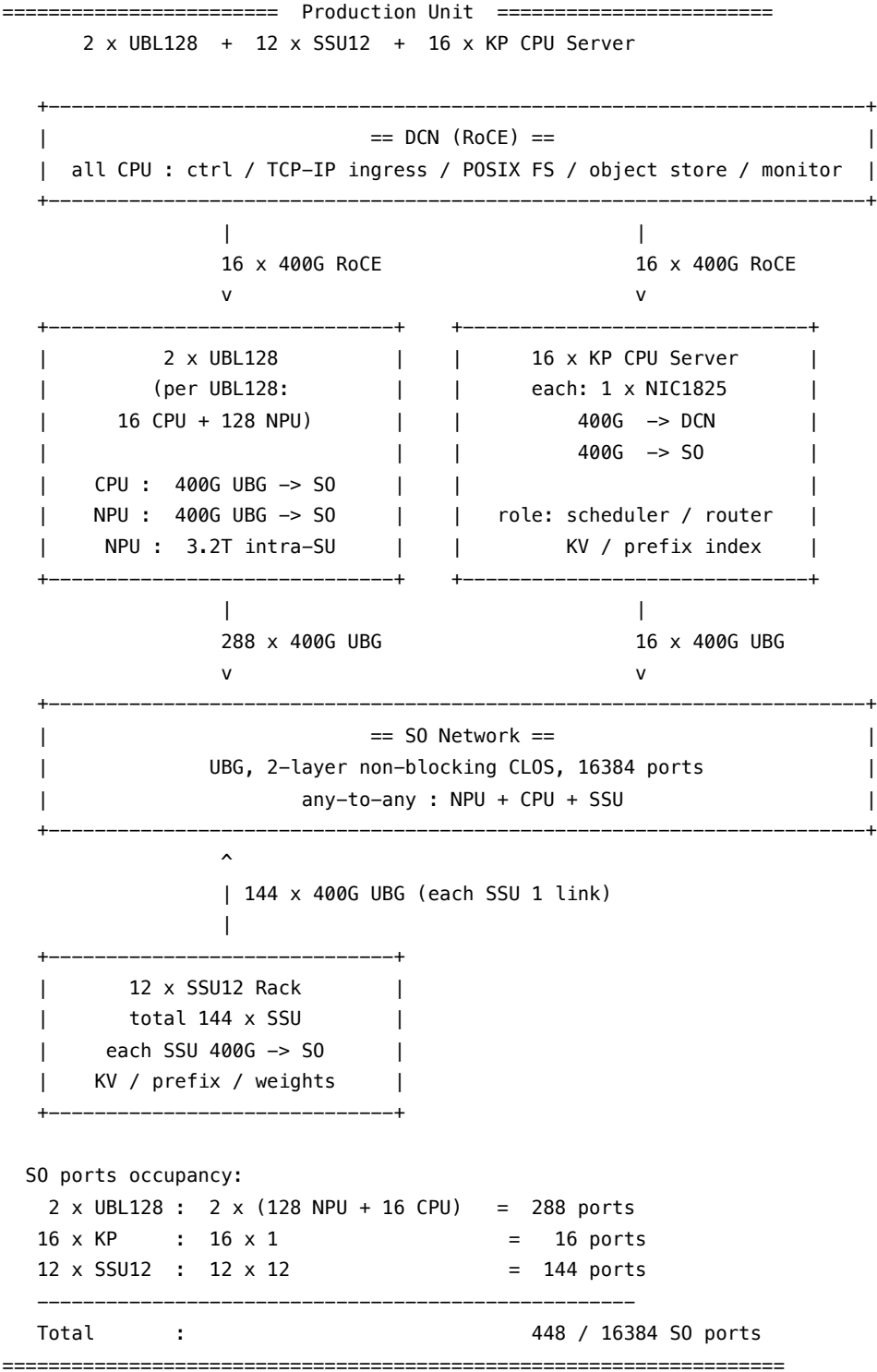
=====

1.4.4 最小生产部署单元 = 2 x UBL128 + 12 x SSU12 + 16 x 鲲鹏 CPU 服务器

鲲鹏 CPU 服务器 (KP CPU Server): 通用 ARM CPU 服务器，每台配 1 张 1825 网卡，该网卡同时提供:

- 1 x 400 Gbps 接入 DCN (RoCE)
- 1 x 400 Gbps 接入 SO 网络 (UBG)

在系统中承担 调度 / Router / KV 元数据管理 / 控制面服务 / 长期 prefix index 等 CPU 侧职责。



1.5 关键带宽与边界总结

通信路径	经过的网络	单链路带宽	备注
同 PC16、同 NUMA 组内 CPU↔NPU	服务器内 UB (一对一直连)	400 Gbps (每对) / 3.2 Tbps (CPU 总)	CPU 主存↔NPU HBM 高带宽通路
同 PC16、同 8-NPU 组内 NPU↔NPU	服务器内 full mesh	400 Gbps	不经过任何交换机
同 PC16、跨 8-NPU 组 NPU↔NPU	SU 交换机	3.2 Tbps (链路上限)	走域内单层交换
同 UBL128、跨服务器 NPU↔NPU	SU 交换机	3.2 Tbps (链路上限)	单跳交换
跨 UBL128 NPU↔NPU / CPU↔NPU	SO (UBG CLOS)	400 Gbps	两层 CLOS、无收敛
CPU↔外部服务 (控制面/HTTP)	DCN (RoCE)	400 Gbps/server	仅 CPU 可见

设计含义：UBL128 内部具备 **3.2 Tbps × 128** 的极高带宽，是 prefill 大算力聚合 / decode 张量并行 / 域内 KV 共享的天然边界；SO 网络（400 Gbps × 任意端口、any-to-any）则用于 **跨 HBD 的 KV 迁移、prefill→decode 解耦、prefix cache 跨域共享**；DCN 仅承担控制面与外部入口，不承担数据面流量。

1.6 支持的硬件场景（验证目标）

我们 **重点验证三档硬件规模**。三档共享上文的硬件抽象（PC16 / 鲲鹏 / SSU12 / UBL128）和网络分层（SU / SO / DCN），仅在 **prefill 集群、decode 集群、KV 持久化存储** 三个维度的台数上做线性放大；**接入服务器（1 台鲲鹏）** 与 **KV cache Meta server（1 台鲲鹏）** 在三档中保持不变。

1.6.1 场景 A —— 小规模（PoC / 端到端功能验证）

角色	硬件	NPU 数	备注
Prefill 集群	1 × PC16	16 NPU	单服务器；NPU↔NPU 走服务器内 full mesh + 同 UBL128 SU 交换机
Decode 集群	1 × PC16	16 NPU	单服务器；同上
服务接入服务器	1 × 鲲鹏 CPU 服务器	—	DCN 入口（外部 HTTP/gRPC）+ SO 控制面（uRPC over UB Urma）+ tokenize / detokenize （v1 集中实现，见 §5.8）+ 调度器
KV cache Meta server	1 × 鲲鹏 CPU 服务器	—	KV 块 / Prefix 索引、调度元数据
KV cache 持久化存储	1 × SSU12（12 × SSU）	—	12 × 400 Gbps UBG → SO

- 总 NPU：**32**
- SO 端口占用： $2 \times \text{PC16} \times (16 \text{ NPU} + 2 \text{ CPU}) + 2 \times \text{鲲鹏} \times 1 + 1 \times \text{SSU12} \times 12 = \mathbf{50 / 16384}$
- 验证目标：端到端功能链路（接入 → 调度 → prefill → KV 迁移 → decode → 输出），KV / Prefix 元数据正确性；**不承担大规模并行验证**

1.6.2 场景 B —— 中规模（跨服务器并行验证）

角色	硬件	NPU 数	与场景 A 的差异
Prefill 集群	4 × PC16	64 NPU	4 台服务器组成跨服务器并行域（共享同一 UBL128 SU 交换机）
Decode 集群	4 × PC16	64 NPU	同上
服务接入服务器	1 × 鲲鹏 CPU 服务器	—	不变
KV cache Meta server	1 × 鲲鹏 CPU 服务器	—	不变
KV cache 持久化存储	1 × SSU12（12 × SSU）	—	不变

- 总 NPU：128
- SO 端口占用： $8 \times \text{PC16} \times 18 + 2 \times \text{鲲鹏} \times 1 + 1 \times \text{SSU12} \times 12 = 158 / 16384$
- 验证目标：在 prefill / decode 各自的 4-PC16 域内验证 跨服务器 SU 并行（TP / SP / EP），以及 prefill → decode 通过 SO 的 KV 推送；存储侧仍为 1 × SSU12，可暴露存储对中等并行的支撑边界

1.6.3 场景 C —— 大规模（满 UBL128 + 满存储框）

角色	硬件	NPU 数	与场景 B 的差异
Prefill 集群	1 × UBL128（= 8 × PC16）	128 NPU	升级到一个完整 UBL128 HBD，独占一个 SU 交换机
Decode 集群	1 × UBL128（= 8 × PC16）	128 NPU	升级到一个完整 UBL128 HBD，独占一个 SU 交换机
服务接入服务器	1 × 鲲鹏 CPU 服务器	—	不变
KV cache Meta server	1 × 鲲鹏 CPU 服务器	—	不变
KV cache 持久化存储	12 × SSU12（144 × SSU）	—	扩容到整列存储框，匹配 256 NPU 的 KV 持久化吞吐

- 总 NPU：256
- SO 端口占用： $2 \times \text{UBL128} \times (128 \text{ NPU} + 16 \text{ CPU}) + 2 \times \text{鲲鹏} \times 1 + 12 \times \text{SSU12} \times 12 = 434 / 16384$
- 验证目标：prefill / decode 各占满一个 UBL128 SU 域时的端到端性能上限、跨域 KV 迁移带宽、跨域 Prefix 共享；KV 持久化扩容到 12 × SSU12（共 144 SSU × 400 Gbps = 57.6 Tbps）以避免存储成为瓶颈

1.6.4 三档对照

维度	场景 A	场景 B	场景 C
Prefill 规模	1 × PC16（16 NPU）	4 × PC16（64 NPU）	1 × UBL128（128 NPU）
Decode 规模	1 × PC16（16 NPU）	4 × PC16（64 NPU）	1 × UBL128（128 NPU）
SU 域并行规模	服务器内 / 16 NPU	跨服务器 / 64 NPU	满 UBL128 / 128 NPU
接入服务器	1 × 鲲鹏	1 × 鲲鹏	1 × 鲲鹏
KV Meta server	1 × 鲲鹏	1 × 鲲鹏	1 × 鲲鹏
KV 持久化存储	1 × SSU12（12 SSU）	1 × SSU12（12 SSU）	12 × SSU12（144 SSU）
总 NPU	32	128	256
SO 端口占用	50	158	434
主要验证侧重	端到端功能 / 控制面	跨服务器 SU 并行 / KV 路径	满 UBL128 性能上限 / 满存储吞吐

递进关系：A → B 关注从单服务器到跨服务器跨 SU 域的扩展；B → C 关注从局部 UBL128（4 PC16）到满 UBL128（8 PC16）的扩展，并同步把存储侧从 1 × SSU12 扩到 12 × SSU12 以匹配算力侧 8× 的放大。三档共享同一套软件路径（解耦调度、KV 元数据、Prefix index、SO 数据面、DCN 控制面），仅在硬件规模上做线性放大；接入与 Meta server 始终各为 1 台鲲鹏。

1.7 DCN 上的基础设施服务（KV cache 之外）

DCN（RoCE）的角色不仅是 §1.3 中的"控制面 + 对外入口"。系统进一步**假设 DCN 上预先部署了两类基础设施服务**，并对所有 CPU（PC16 内的 CPU + 鲲鹏 CPU 服务器的 CPU）开放：

1.7.1 POSIX-compliant 分布式文件系统（承载 KV cache 之外的所有数据）

- **形态：**NFS / 3FS / 类 Amazon S3 对象存储 等任一具备 **POSIX 语义**（或 S3 兼容 GET/PUT 语义）的分布式存储均可。
- **接入面：**所有 PC16 内 CPU 与所有鲲鹏 CPU 服务器，通过 DCN（RoCE 或 TCP）挂载/访问该分布式 FS。**NPU 不直接挂载：**所有数据搬运经 CPU 中转（CPU 通过 DCN 拉取 → 服务器内 8 × 400 Gbps UB → NPU HBM）。
- **承载内容（一切非 KV 持久化数据）：**
 - 模型权重（model checkpoint，加载到 NPU HBM 之前的源端）、tokenizer / 词表 / safety 模型 / embedding 表
 - prompt 语料、评测数据集、推理副本/重放数据
 - 日志归档、metrics 长期存储、监控快照、追踪
 - 配置文件（包括 §2.5 的部署 YAML 三档）、版本元数据、镜像中间产物
- **与 SSU12 的边界划分：**

存储面	物理位置	承载网络	承载内容	与算力的耦合
SSU12	SO 平面	UBG CLOS	KV cache 数据面（专用） —— KV 块、prefix block	紧耦合，按 §1.6 三档线性扩缩
DCN POSIX FS / 对象存储	DCN 平面	RoCE	KV 之外的所有持久化数据	与算力侧解耦，可独立扩缩

两个存储面 **完全独立**：SSU12 不要求 POSIX 语义；POSIX FS 不参与 KV 块的运行时路径。

1.7.2 TCP/IP 接入网（承载外部 IP 世界的对接）

- **形态：**DCN 之上承载标准的 TCP/IP 子网，提供与"外部 IP 世界"的通用对接。
- **承载内容：**
 - 用户请求接入（HTTP / gRPC，由前端的 1 台鲲鹏接入服务器对外暴露）
 - 镜像仓库 / 软件包仓库拉取
 - 外部监控 / 日志聚合后端（Prometheus / Loki / OpenTelemetry collector 等）
 - 外部数据集拉取、远程鉴权 / 配额服务
- **与 SO 的边界：**SO（UBG CLOS）严格保留给 NPU/CPU 数据面（KV 迁移、跨 UBL128 张量、SSU 块拉取），**不承载 TCP/IP**；TCP/IP 仅在 DCN 上存在。

1.7.3 三类网络职责再总结（含 DCN 上的服务）

网络	协议	接入对象	承载流量
SU （UBL128 内）	UB	NPU only	HBD 内张量并行 / 紧耦合集合通信
SO （跨 UBL128）	UBG, 2 层无收敛 CLOS	NPU + CPU + SSU	KV 迁移 / 跨域并行 / 持久化 KV 访问； 不走 TCP/IP
DCN	RoCE	CPU only （PC16 CPU + 鲲鹏 CPU）	控制面 + POSIX 分布式 FS （非 KV 数据） + TCP/IP 接入网 （外部 IP 世界）

设计含义：把 *KV cache* 与"其它一切持久化数据"严格分到 **两个独立存储面**（SO 上的 SSU12 vs DCN 上的 POSIX FS），可在 §1.6 三档之间各自独立扩缩。例如：场景 C 把 SSU12 拉满至 12 框以匹配 256 NPU 的 KV 持久化吞吐，但 DCN 上的 POSIX FS 实例数和容量可与场景 A 保持一致（模型权重 / 配置 / 日志归档的需求并不随 NPU 数 8× 放大）；反之亦然。这种解耦让 *算力侧扩缩* 与 *运维数据侧扩缩* 完全独立、互不绑定。

2. 软件设计原则：一套代码、配置驱动适配三档硬件

2.1 核心要求

一份源码、一份编译产物（按角色分发：**prefill-worker / decode-worker / frontend / kv-meta / kv-store-agent** 等），通过 *仅修改配置文件* 即可适配 §1.6 的场景 A / B / C 三档硬件规模。任何硬件规模差异都不得通过分支版本、专用编译宏或针对某一档的代码路径来表达。

这条要求是 §1.6 三档"线性放大"的软件对应物：硬件上是 PC16 → 4 × PC16 → UBL128，存储上是 1 × SSU12 → 1 × SSU12 → 12 × SSU12；软件上对应的就是 *同一套二进制 + 不同YAML*。

2.2 必须由配置驱动的维度

凡是在 §1.6 三档之间发生变化的维度，**全部** 抽出来作为配置项；代码内部对这些维度做参数化（典型上以 `sharded_tensor` 的 `shard_shape × rank_shape` + 角色拓扑表达，参见 `sharded_tensor.md`）。

维度	配置项（示意）	场景 A	场景 B	场景 C
Prefill 算力规模	<code>prefill.scope / prefill.servers</code>	pc16 / 1	ubl128_partial / 4	ubl128_full / 8
Decode 算力规模	<code>decode.scope / decode.servers</code>	pc16 / 1	ubl128_partial / 4	ubl128_full / 8
Prefill 并行策略	<code>prefill.parallel = {tp, sp, ep, dp}</code> （即 <code>rank_shape</code> 各维度）	由 <code>servers × 16 NPU</code> 推导上限	同左	同左
Decode 并行策略	<code>decode.parallel = {tp, sp, ep, dp}</code>	同上	同上	同上
KV 持久化存储规模	<code>kv_store.ssu12_count</code>	1	1	12
接入服务器实例数	<code>frontend.instances</code>	1	1	1
KV Meta server 实例数	<code>kv_meta.instances</code>	1	1	1
SU 域跨度	由 <code>*.scope</code> 推导（同 PC16 / 同 UBL128 / 跨 UBL128）；运行时透明	同 PC16 内	同 UBL128 内（4/8 占位）	同 UBL128 内（满）
SO 节点发现	由 SO 注册中心 / Meta server 动态拉取	三档共享	三档共享	三档共享

2.3 不允许在代码中出现的硬编码

- 不得 出现常量化的 NPU 数 / 服务器数（如 `NUM_NPU = 128`、`NUM_PREFILL_SERVERS = 4`）。
- 不得 出现常量化的并行度（如 `TP = 8`、`EP = 8`）。所有并行度必须从配置 → `rank_shape` 推导。
- 不得 出现以"是否跨 UBL128 / 是否有 SU 交换机 / SSU12 数量"为条件的业务分支。这类拓扑差异应在 **SO 数据面 + 元数据层** 内被透明处理，业务层只看到统一的"按 KV 块 / 按 prefix block 寻址"的接口。

- **不得** 在源码中写死 SSU 的 IP / 端口、KV 存储的 shard 数；这些必须来自 KV Meta server 在启动时下发的拓扑表。
- **不得** 假设 prefill 与 decode 处于相同规模（虽然 §1.6 三档目前是对称的，软件不应依赖这一对称性）。

2.4 跨三档不变的软件骨架

下列内容在 A / B / C 三档中 **完全相同**，是"一套代码"的实质内容：

- **角色集**：prefill-worker、decode-worker、frontend、kv-meta、kv-store-agent。三档都只有这五类角色，仅每个角色的实例数和并行度不同。
- **数据面接口**：SO 上的 KV 块 push/pull、Prefix block 拉取、cross-rank sharded_tensor 访问 —— 接口签名、序列化协议、流控策略在三档完全一致。
- **控制面接口**：DCN 上的请求接入、调度路由、心跳、监控、配置下发 —— 接口签名在三档完全一致。
- **元数据模型**：KV 块索引、Prefix 索引、请求路由表、租户/quota 表 —— schema 在三档完全一致。
- **可观测性 / 容错**：日志埋点、追踪、重试/熔断策略、KV 块校验机制 —— 与硬件规模无关。

2.5 三档配置草图（示意）

仅展示"由配置驱动"的关键差异，不是最终 schema。最终 schema 待详细设计阶段确定。

```
# 场景 A — 1 + 1 + 1 + 1 + 1
prefill: { scope: pc16,          servers: 1, parallel: { tp: 8,  sp: 1, ep: 1, dp: 2 } }
decode:  { scope: pc16,          servers: 1, parallel: { tp: 8,  sp: 1, ep: 1, dp: 2 } }
frontend: { instances: 1, host: kp }
kv_meta:  { instances: 1, host: kp }
kv_store: { ssu12_count: 1 }
```

```
# 场景 B — 4 + 4 + 1 + 1 + 1
prefill: { scope: ubl128_partial, servers: 4, parallel: { tp: 16, sp: 1, ep: 1, dp: 4 } }
decode:  { scope: ubl128_partial, servers: 4, parallel: { tp: 8,  sp: 1, ep: 1, dp: 8 } }
frontend: { instances: 1, host: kp }
kv_meta:  { instances: 1, host: kp }
kv_store: { ssu12_count: 1 }
```

```
# 场景 C — UBL128 + UBL128 + 1 + 1 + 12
prefill: { scope: ubl128_full,    servers: 8, parallel: { tp: 16, sp: 1, ep: 8, dp: 1 } }
decode:  { scope: ubl128_full,    servers: 8, parallel: { tp: 16, sp: 1, ep: 1, dp: 8 } }
frontend: { instances: 1, host: kp }
kv_meta:  { instances: 1, host: kp }
kv_store: { ssu12_count: 12 }
```

2.6 该原则下的验收标准

- **同一 git commit、同一 build artifact** 必须能在三档硬件上分别正确启动；切换场景仅替换 YAML，不重编译。
- 三档启动后的业务路径（接入 → 路由 → prefill → KV 迁移 → decode → Prefix 复用）功能等价、协议兼容、监控指标对齐。
- 代码静态扫描中 **不得** 出现以硬件规模为条件的 if-else（违例视为该原则的硬性破坏，需重构而非添加新分支）。
- 新增功能时必须同步在三档配置上跑通；任何只在某一档可用的功能不视为"完成"。

3. KV Cache 设计

3.1 总体方案

决策点	选择	备注
存储介质	SSU （来自 §1.6 的 SSU12 框列），经 SO (UBG CLOS) 读写	与算力侧（NPU / CPU）解耦；不参与 DCN 上的 POSIX FS (§1.7)
存储抽象	File-system-free ：直接以 SSU LBA （Logical Block Address）寻址	不构建 inode / 路径解析层；每次 IO 是 (ssu_id, lba, len) 三元组
元数据服务	1 台鲲鹏 CPU 服务器 （即 §1.6 中 kv_meta）维护 prefix → KV block 映射	单实例；高可用留作后续扩展
索引结构	Prefix radix tree (patricia trie)，节点键为 token chunk 的哈希	支持最长前缀匹配
索引粒度	固定 token chunk （默认 64 token / chunk）；不按单 token 组织	见 §3.5
物理布局	以 (prefix_id, layer_id, chunk_id) 为最小分配单元，stripe 到所有 SSU	见 §3.4
副本 / 故障恢复	v1 不实现 （replication_factor = 1）	留作后续扩展，§3.7
写时序	prefill 计算完一个 chunk → 写 SSU → 通知 meta server 入树	meta 入树后该 prefix 才对后续请求可见

3.2 LBA-direct 存储模型（无文件系统）

每个 SSU 暴露一段连续的 LBA 空间（0 .. capacity / lba_size - 1）。系统启动时由 KV Meta server 把每个 SSU 的 LBA 空间切成 **allocation pool**（如固定大小的 extent），运行时按需分配。

- **没有 inode、目录、路径**：(ssu_id, lba_start, lba_count) 即定位一段 KV 数据
- **NPU 不直接寻址 SSU**：所有 KV 读写经 PC16 内 CPU（或某个专用 KV agent）发起 SO 请求，CPU/agent 充当 LBA 客户端
- **空间回收**：基于引用计数（同一 prefix 多次命中时引用 +1，请求结束 -1，到 0 后块进入回收池）；回收池按 LBA 段维度、不做碎片合并（v1）
- **优点**：免去文件系统开销；写路径长度短（NPU HBM → CPU 主存 → SO → SSU LBA），读路径对称；LBA 是已知物理拓扑量，对 SSU 自身的 GC / wear-leveling 更友好
- **代价**：所有元数据正确性必须由 KV Meta server 单点保证（v1 接受单实例）；因此 §3.6 的 meta server 是关键路径

3.3 KV Cache 单元尺寸（DeepSeek V4 Hybrid Attention）

关键变化：V4 系列（Pro / Flash）不再使用 V3 的 **MLA**；改用 **Hybrid Attention** 架构 —— 在 *序列维* 做激进压缩 + 稀疏检索 + 滑动窗口，而不是像 MLA 那样在 *特征维* 做低秩压缩。这一架构在 1 M token 长上下文下相对 V3.2 的总 KV 缩减为 **V4 Pro: 10 % (-90 %) / V4 Flash: 7 % (-93 %)**，把 SSU 持久化 KV 的可行性从"勉强"推到"宽裕"。

3.3.1 V4 Hybrid Attention 架构要点

V4 在每个 transformer block 内 **交替** 部署两种压缩注意力，并叠加一个共用的滑动窗口分量：

组件	序列压缩比 (m)	注意力计算方式	主要作用
CSA —— Compressed Sparse Attention	每 m = 4 个连续 token，用学习得到的 softmax 权重压成 1 个 compressed KV entry	稀疏 ：用 Lightning Indexer (FP4 4-bit) 打分，选 top-k 最相关的 compressed entries 后做 attention (DSA)	中等压缩 + 选择性检索；保留细粒度 token 间关系
HCA —— Heavily Compressed Attention	每 m = 128 个连续 token，压成 1 个 compressed KV entry	稠密 ：因为序列被压得很短，可以在所有 compressed entries 上做 dense attention	极致压缩；以远低于 V3 的代价支撑超长上下文
Sliding Window (与 CSA / HCA 叠加)	最近 W = 128 个 token 保持 未压缩 原始 K/V	与对应层的压缩输出叠加	不丢失局部细节 (最近上下文)
Lightning Indexer (仅 CSA 路径用)	每个 compressed entry 生成一个 FP4 indexer key	用 query 与 indexer key 做相似度打分以选 top-k	检索路径的存储与计算大幅省

整网 L 层中按一定比例排布 CSA / HCA 层 (混合比例由模型设计决定，**v1** 设计期视为可配置)。

3.3.2 KV cache 在 SSU 上的存储成本 (按层类型分别核算)

把 SSU 上需要存的内容拆成三类：

存储项	公式 (每层每 prompt)	量级 (在 1 M token 上)
(a) CSA 层 compressed entries	$\lceil (N - W) / m_{\text{csa}} \rceil \times \text{csa_entry_bytes}$	$\lceil (1\text{M}-128)/4 \rceil \approx 250 \text{ K entries} / \text{CSA 层}$
(b) HCA 层 compressed entries	$\lceil (N - W) / m_{\text{hca}} \rceil \times \text{hca_entry_bytes}$	$\lceil (1\text{M}-128)/128 \rceil \approx 7\,800 \text{ entries} / \text{HCA 层}$
(c) 滑动窗口 (每层)	$W \times \text{window_token_bytes}$	$128 \times \sim 1 \text{ KiB} \approx 144 \text{ KiB} / \text{层}$ (与 N 无关, 常数)
(d) Lightning Indexer (仅 CSA 层)	$\lceil (N - W) / m_{\text{csa}} \rceil \times \text{indexer_per_entry}$	$250 \text{ K} \times 32 \text{ B} = 8 \text{ MiB} / \text{CSA 层}$ (在 1 M 下, FP4 极小)

单 entry 字节数 (占位估计)：

量	V4 Pro	V4 Flash	说明
csa_entry_bytes	~1 152 B (占位)	~896 B (占位)	与 V3 单 token MLA latent 同量级；待 V4 官方规格替换
hca_entry_bytes	~1 152 B (占位)	~896 B (占位)	同上
window_token_bytes	~1 152 B (占位)	~896 B (占位)	滑动窗口里的原始 K/V, bf16
indexer_per_entry (FP4)	~32 B (占位)	~32 B (占位)	head_dim 量级 $\times 0.5 \text{ B} / \text{元素}$

占位值取自 **V3 MLA 同量级**作为保守上界。V4 序列压缩节省的根本来源是 *token* 数从 *N* 缩到 *N/m*，所以即使每 entry 字节数不变，总 KV 也按 1/m 缩；这是 §3.3.3 比例的来源。

3.3.3 平均每 raw-token KV cache (与用户引用的 1 M 比例一致)

定义"每 raw token 的有效 KV 占用"为 总 KV / 总 raw token 数。在 1 M 上下文且滑动窗口占比可忽略下：

avg_per_token_kv ≈
(L_csa × csa_entry_bytes / m_csa)
+ (L_hca × hca_entry_bytes / m_hca)
+ (L_csa × indexer_per_entry / m_csa)
+ (L × W × window_token_bytes / N)

FP4 indexer
1/N 衰减, N 大时可忽略

直接采用用户给出的 V4 vs V3.2 比例作为标定：

模型	V4 vs V3.2 (1 M context)	反推平均每 raw token KV	备注
V4 Pro	10 %	V3.2 ≈ 70 KiB → V4 Pro ≈ 7 KiB / token	−90 % 减少
V4 Flash	7 %	V3.2 ≈ 70 KiB → V4 Flash ≈ 5 KiB / token	−93 % 减少

对比 §3.3 旧版（误认为 V4 仍是 MLA）的 90 KiB / 28 KiB，新架构再砍 **~13× / ~6×**。

3.3.4 不同 prompt 长度下的总量（V4 新架构）

Prompt 长度	V4 Pro (≈ 7 KiB / token)	V4 Flash (≈ 5 KiB / token)
8 K tokens	~56 MiB	~40 MiB
32 K tokens	~224 MiB	~160 MiB
128 K tokens	~896 MiB	~640 MiB
1 M tokens	~7.0 GiB	~5.0 GiB

3.3.5 SSU 总容量匹配（按 1 M context 折算）

场景	SSU 数	总容量（取 C_ssu = 7.68 TB）	V4 Pro 1 M-prompt 容纳	V4 Flash 1 M-prompt 容纳
A / B	12	~92 TiB	~13 K 个	~18 K 个
C	144	~1.1 PiB	~157 K 个	~220 K 个

设计含义：V4 Hybrid Attention 把"持久化 1 M 长上下文 KV cache"从一个挑战变成轻负担。即使在场景 A（12 SSU）下也能容纳数千个 1 M token 活跃 prompt；场景 C 容量提高一个量级。

3.3.6 V4 各层"chunk 内 KV 字节"小结（chunk_tokens = 512, §3.5）

V4 chunk 必须能整除 max(m_hca, W) = 128。下表用推荐值 chunk_tokens = 512：

层类型	每 chunk 内 entry 数	每 chunk 字节数（V4 Pro, ~1152 B/entry）	4 KiB LBA 对齐
CSA 层 entries	512 / 4 = 128	128 × 1 152 = 144 KiB	36 LBA, 整数倍
HCA 层 entries	512 / 128 = 4	4 × 1 152 = 4.5 KiB	2 LBA（含 12.5 % padding）
CSA 层 indexer（FP4）	128 entries	128 × 32 = 4 KiB	1 LBA, 整数倍
滑动窗口（每层）	W = 128 raw tokens（仅一份，不参与 chunk 化）	128 × 1 152 = 144 KiB / 层	单独存储

关键观察：CSA 层 chunk 与 HCA 层 chunk 字节数相差 **32x**（V4 Pro 下 144 KiB vs 4.5 KiB），所以 §3.4 的 stripe 分配单元必须以 (prefix, layer, chunk) 为粒度，不能假设各层 **chunk** 等大；不同 layer 类型独立选 SSU 与 LBA。

3.4 多层 KV 在 SSU 上的物理排布

3.4.1 关键问题：61 层是否同放一个 SSU？

结论：不放在同一 SSU。以 (prefix, layer, chunk) 为分配单元 stripe 到所有 SSU。

3.4.2 三个候选方案对比

方案	分配单元	单元大小 (chunk=512, V4 Pro)	单 prefix 命中时的 SSU 行为
(a) token-major：单 token 全 L 层连放	(prefix, token)	~7 KiB（V4 Pro 平均）	每 token 一次跨层小 IO；放大严重；HCA 层完全不适合
(b) chunk-all-layers-on-one-SSU：— chunk 的全 L 层放同一 SSU	(prefix, chunk)	CSA 层 144 KiB × L_csa + HCA 层 4.5 KiB × L_hca + 滑动窗口 → 单 prefix — chunk ≈ 几 MiB	一个 chunk 内的所有 PP stage 都打到同一 SSU → 串行化；并行价值丢失
(c) per-(layer-type, chunk) striping (推荐)	(prefix, layer_id, chunk_id)	CSA 层 ：144 KiB（36 LBA）； HCA 层 ：4.5 KiB（2 LBA, 12.5 % padding）； indexer ：4 KiB（1 LBA）	同一 chunk 的 L 层分布到 ~L 个不同 SSU → PP stage 自然并行；CSA / HCA / indexer 各自是独立的小分配单元

为什么不选 (b)：

- 流水线并行（PP，按层切分到不同 NPU 上）下每个 PP stage 同时需要“自己那几层”的 KV
- 若一个 chunk 的 L 层挤在一个 SSU 上，PP fan-out 的 L 路并发请求都打向同一 SSU → 该 SSU 的 400 Gbps 链路成为瓶颈
- 方案 (c) 把 L 路并发分到 L 个不同 SSU

为什么不选 (a)：

- HCA 层每 raw token 仅贡献 ~9 B（V4 Pro，1152 / 128）→ token-major IO 比 LBA 还小 100x，灾难性放大
- 索引也退化为 token-级（见 §3.5）

3.4.3 推荐布局（方案 c）的关键点

- 分配单元粒度 = (prefix_id, layer_id, chunk_id)；同 prefix 内根据 layer_id 解析层类型（CSA / HCA），各自分配
 - CSA 层 chunk**（V4 Pro, chunk=512）：512 / 4 × 1 152 = **144 KiB**（36 × 4 KiB LBA，整数倍）
 - HCA 层 chunk**（V4 Pro, chunk=512）：512 / 128 × 1 152 = **4.5 KiB**（2 × 4 KiB LBA，含 12.5 % padding）
 - CSA 层 indexer chunk**（V4 Pro）：512 / 4 × 32 = **4 KiB**（1 × 4 KiB LBA，整数倍；与 entry chunk 同 SSU 协同放置以减少二次 SO 跳转）
 - V4 Flash 数值按 entry_bytes ≈ 896 B 同比例缩
- stripe 函数** = ssu_id = hash(prefix_id, layer_id, chunk_id) mod N_ssu
 - 相同 (prefix, chunk) 不同 layer → 不同 SSU（PP 并行友好）
 - CSA entry chunk 与其 indexer chunk **建议放同 SSU**（用同一 hash key 决定）以避免索引-取数跨 SSU
- 滑动窗口**（每层 W=128 raw tokens，~144 KiB / 层）独立分配，不参与 chunk 化；可整 prefix-level 一次写入与重读
- 单 prefix 全量**：
 - V4 Pro 1 M prompt：80 layers × [1 M / 512] ≈ 80 × 2 048 = **~164 K 个 entry chunk** + 滑动窗口 + indexer → 总 ~7 GiB
 - V4 Flash 1 M prompt：32 × 2 048 ≈ **~66 K 个 entry chunk** → 总 ~5 GiB

3.4.4 带宽校核

场景 C：144 SSU × 400 Gbps = **57.6 Tbps** 聚合。

- V4 Pro 1 M prompt 全量恢复 (~7 GiB) = 7 GiB × 8 / 57.6 Tbps ≈ **1.0 ms** (理想)，瓶颈在 NPU SO 接收 (102.4 Tbps) → 实际 ~5 ms 以内
- V4 Flash 1 M prompt (~5 GiB) ≈ **0.7 ms** 理想，~3 ms 实际
- 与 V3 MLA 同样 prompt 长度的恢复时间相比快约 **10x** (KV 总量 1/10)；Hybrid Attention 让 1 M context 实时召回成为可能

3.4.5 每 stripe 单元的精确字节数

按 §3.4.3 的方案 (c), stripe 单元 = (prefix_id, layer_id, chunk_id)。但因 V4 各 layer 类型存储成本不同 (CSA / HCA / 滑动窗口) 以及 CSA 层额外有 indexer, **stripe 单元的字节数有四种**：

stripe 子类型	何时出现	公式	V4 Pro (entry ≈ 1 152 B)	V4 Flash (entry ≈ 896 B)	LBA 对齐 (4 KiB LBA)
CSA entry chunk	layer_id 是 CSA 层	$chunk_tokens / m_csa \times csa_entry_bytes = 128 \times entry_bytes$	144 KiB (128 × 1 152 = 147 456 B)	112 KiB (128 × 896 = 114 688 B)	整数倍 (Pro 36 LBA / Flash 28 LBA)
CSA indexer chunk (FP4)	layer_id 是 CSA 层 (与 entry chunk 同 SSU 连续 LBA, 作为一段 148 / 116 KiB 单 IO, 见 §3.4.8 Level 1)	$chunk_tokens / m_csa \times indexer_per_entry = 128 \times 32$	4 KiB (4 096 B)	4 KiB (同 Pro, 与 entry_bytes 无关)	整数倍 (1 LBA)
HCA entry chunk	layer_id 是 HCA 层	$chunk_tokens / m_hca \times hca_entry_bytes = 4 \times entry_bytes$	4.5 KiB (4 × 1 152 = 4 608 B)	3.5 KiB (4 × 896 = 3 584 B)	Pro 2 LBA (含 12.5 % padding) / Flash 1 LBA (含 12.5 % padding)
滑动窗口 (每层一份, 不分 chunk)	每个 prefix 末尾一次写入; 不参与 chunk 流	$W \times window_token_bytes = 128 \times entry_bytes$	144 KiB / 层 (128 × 1 152)	112 KiB / 层 (128 × 896)	整数倍

关键观察：

- **CSA entry stripe 与 HCA entry stripe 字节相差 32x** (V4 Pro 144 KiB vs 4.5 KiB) —— 这正是序列压缩比 $m_hca / m_csa = 128 / 4$ 的体现；分配器必须按 layer_type 走两条不同路径。
- **CSA 层一次完整分配 = entry stripe + indexer stripe, 同 SSU 连续 LBA** (默认 Level 1 拼接, 详见 §3.4.8), 逻辑上视作一个二元 allocation 但物理上只占 1 段 LBA、1 次 IO：

- V4 Pro: $144 + 4 = 148 \text{ KiB}$ (**37 LBA**) / CSA layer / chunk
- V4 Flash: $112 + 4 = 116 \text{ KiB}$ (**29 LBA**) / CSA layer / chunk
- **HCA chunk 偏小** (4.5 / 3.5 KiB) 是 chunk_tokens=512 的折中代价 (12.5 % padding)；若要消除可升 chunk_tokens 到 4096 (HCA 32 entry / chunk = 36 KiB, 整数 LBA), 但索引粒度会粗 8×。当前选 512 优先保索引细度。
- **滑动窗口**单独存储, 每个 prefix 仅 $L \times 144 \text{ KiB}$ (V4 Pro 80 层 $\rightarrow 11 \text{ MiB}$ / prefix; V4 Flash 32 层 $\rightarrow 3.5 \text{ MiB}$ / prefix), 与 N 无关; 写时序也不同 (在 prefix 收尾时一次性写)。

3.4.6 单 prefix 全量字节估算 (V4 Pro 1 M context)

按 §3.3.3 反推的 V4 Pro 层混比 (约 **22 CSA + 58 HCA = 80 层**, 由 7 KiB / token 平均反推):

$$\text{chunks_per_prefix} = \lceil \text{prefix_len} / \text{chunk_tokens} \rceil = \lceil 1 \text{ M} / 512 \rceil = 2\ 048$$

子项	公式	V4 Pro (1 M context)	V4 Flash (1 M context, ~10 CSA + 22 HCA = 32 层 假设)
CSA entry stripes	$L_{\text{csa}} \times \text{chunks} \times 144 \text{ KiB}$	$22 \times 2\ 048 \times 144 \text{ KiB} \approx \mathbf{6.19 \text{ GiB}}$	$10 \times 2\ 048 \times 112 \text{ KiB} \approx \mathbf{2.19 \text{ GiB}}$
CSA indexer stripes	$L_{\text{csa}} \times \text{chunks} \times 4 \text{ KiB}$	$22 \times 2\ 048 \times 4 \text{ KiB} \approx \mathbf{176 \text{ MiB}}$	$10 \times 2\ 048 \times 4 \text{ KiB} \approx \mathbf{80 \text{ MiB}}$
HCA entry stripes	$L_{\text{hca}} \times \text{chunks} \times 4.5 \text{ KiB}$ (V4 Pro) / $\times 3.5 \text{ KiB}$ (V4 Flash)	$58 \times 2\ 048 \times 4.5 \text{ KiB} \approx \mathbf{522 \text{ MiB}}$	$22 \times 2\ 048 \times 3.5 \text{ KiB} \approx \mathbf{154 \text{ MiB}}$
滑动窗口 (不分 chunk)	$L \times 144 \text{ KiB}$ / $L \times 112 \text{ KiB}$	$80 \times 144 \text{ KiB} \approx \mathbf{11 \text{ MiB}}$	$32 \times 112 \text{ KiB} \approx \mathbf{3.5 \text{ MiB}}$
合计		$\approx \mathbf{6.9 \text{ GiB}}$ (与 §3.3.4 的 ~7 GiB 一致 ✓)	$\approx \mathbf{2.4 \text{ GiB}}$ (注: 与 §3.3.4 的 ~5 GiB 偏离, 需以官方 V4 Flash 层混比为准)

Flash 的总量在此处偏低 于 §3.3.4 的 ~5 GiB —— 原因是: §3.3.4 的"~5 KiB / token"是用户给出的总比; 上表的层混比 (10 + 22) 是粗估。实际取定 $L_{\text{csa}} / L_{\text{hca}}$ 比例后两侧应回到一致。本表的 Pro 比例 (22 + 58) 已与 §3.3.4 校准过。

3.4.7 SSU 上的 stripe 分配粒度小结

项	取值
stripe 寻址 key	(prefix_id, layer_id, chunk_id)
stripe 选 SSU	$\text{ssu_id} = \text{hash}(\text{prefix_id}, \text{layer_id}, \text{chunk_id}) \bmod n_{\text{ssu}}$
stripe 字节范围	4.5 KiB (HCA entry, 最小有效) ~ 148 KiB (CSA entry+indexer 拼段, V4 Pro 最大)
stripe 内字节布局	连续 LBA ; CSA 层 = [entries indexer FP4] 一段 37 LBA (Pro) / 29 LBA (Flash), 1 次 IO 取齐 (Level 1 拼接, §3.4.8)
LBA padding 最大值	HCA 层 12.5 % (4.5 KiB / 5 KiB 的 1 LBA 浪费)
滑动窗口 (非 stripe)	每 prefix 一次性写入 $L \times W \times \text{entry_bytes}$ (V4 Pro 11 MiB / prefix); 每 (prefix, layer) 自身一段连续 LBA

3.4.8 进一步降低 IOPS：两级拼接策略

目标：把 §3.4.5 列出的离散 stripe 子单元 尽可能拼成连续 LBA，从而降低 SSU 端的 IOPS 与随机访问次数；只让 SSU 处理少量大块顺序 IO，把"小块随机读"的子系统压力卸掉。

按拼接尺度，分两级，v1 强制采用 Level 1，Level 2 作为 v1.5 / v2 的可选优化。

Level 1：同一 (layer, chunk) 内的子类型拼接（v1 默认开启）

把同一 stripe 单元（prefix, layer, chunk）内的全部子类型 就地拼成一段连续 LBA：

layer 类型	拼接前（朴素分配）	拼接后（Level 1）	IO 次数
CSA 层 chunk	entry chunk 36 LBA + indexer chunk 1 LBA，可能不相邻 → 2 次 IO	[entries indexer FP4] 共 37 LBA = 148 KiB (Pro) / 116 KiB (Flash) 一段 → 1 次 IO	2 → 1，省 50 %
HCA 层 chunk	entry chunk 2 LBA（Pro，含 12.5 % padding）/ 1 LBA（Flash）	同上（本身就 1 段）	1 → 1
滑动窗口（每 prefix, layer）	W × entry_bytes = 144 KiB（Pro）/ 112 KiB（Flash），独立段	同上，每 (prefix, layer) 一段	1 → 1

实现成本：

- KV Meta server 的 LBA 分配器：CSA 层一次申请 csa_entry_bytes + 4 KiB 的连续段（37 / 29 LBA），返回单个（ssu_id, lba_start, len）三元组而不是两个。
- prefill 写：CSA 层先写 entry，紧接着写 indexer，到同一段的相邻偏移；不需要二次对齐。
- decode 读：1 次 SO IO 读入 148 KiB / 116 KiB，NPU 端按已知偏移切出 entry 与 indexer 两部分。

几乎零成本，每个 CSA 层每 chunk 省 1 次 IO。V4 Pro 1 M context 下 80 层 × 22 CSA × 2 048 chunks = 全 prefix 召回省 ~4.5 万次 IO。

Level 2：跨 chunk、按 (prefix, layer) 流式拼接（v1.5 / v2 升级项）

把同一 prefix、同一 layer 的所有 chunk 在 SSU 上连续顺序排布（prefill 期间作为 append stream），形成一段大顺序 LBA。

维度	§3.4.3 方案 (c) + Level 1	Level 2（per-(prefix, layer) append-stream）
stripe key	(prefix, layer, chunk)（每 chunk 一段 LBA）	(prefix, layer)（每层一段大 LBA，内部按 chunk 顺序排布）
单 prefix 单 layer 的 stripe 段数	2 048（chunk_tokens=512, 1 M context）	1 段 （整层一段，长度 = chunks × per_chunk_bytes）
单 prefix 单 layer 的 IO 次数	2 048	1 次大顺序 IO
整 prefix 全网恢复 IOPS（V4 Pro 80 层）	80 × 2 048 ≈ 164 K IOPS / prefix	80 IOPS / prefix
单次 IO 字节量	148 KiB（CSA 层）/ 4.5 KiB（HCA 层）	~288 MiB（CSA 层全 chunks 拼段）/ ~9 MiB（HCA 层）
总传输字节	相同（KV 总量不变）	相同
IOPS 缩减比	1×	~2 000×

对 SSU 子系统的收益：

NVMe / SSU 的 **小 IOPS** 与 **大块顺序吞吐** 是两个独立瓶颈。Level 1 后单 SSU 在 1 M-context 整 prefix 召回中需要 ~1 100 IOPS (5 ms 内)，Level 2 后只需 ~0.5 IOPS。SSU 的小 IO 队列深度被完全卸掉，给 GC / wear-leveling 留更多空间，长期稳定性显著改善。单次顺序读 ≥ 几十 MiB 几乎一定能跑满 SSU 单链路 400 Gbps。

Level 2 的代价（为什么不在 v1 上）：

- 1. **写入复杂度**：KV Meta server 需为每个 prefill-in-flight 的 (prefix, layer) 维护 append-pointer；prefill 完成后 sealing。
- 2. **prefix 共享语义**：当 prefix B 共享 prefix A 的前 K 个 chunk 时，B 的 KV 是 **两段拼成** —— A 的某 SSU LBA[0..K] + B 自己 append 的尾段。读 B 时为 **2 次顺序 IO**（仍远低于 Level 1 的 chunk 数）。ChunkRecord 必须附加 分段信息（每段 = (ssu_id, lba_start, chunk_count)）。
- 3. **回收复杂度**：当 prefix A 被淘汰但 A 的部分 chunk 仍被 B / C 引用 → 不能整段释放；需要按引用计数细分回收，或采用"写时复制 + 慢回收"。
- 4. **prefill 中途分支 / 乱序写**：同一 prefix 因 in-flight batching 可能出现 chunk 写入顺序 ≠ 逻辑 chunk_id 顺序 → 需要补一层 chunk_index 表把 LBA 偏移映回 chunk_id。
- 5. **不与 PP 友好**：若开启 PP，跨层 fan-out 仍由 stripe key 内 layer_id 维度保证（每 layer 一段，不同 layer 仍 hash 到不同 SSU）；但同 (prefix, layer) 段必须落在单一 SSU，PP stage 在该层的并行度被限制为 1（与 §3.4.2 方案 (b) 的 PP 反例不冲突，因 Level 2 仍按 layer 切到不同 SSU）。

选型矩阵

阶段	拼接级别	实现复杂度	预期收益	推荐
v1	Level 1 (intra-chunk concat: CSA entry + indexer 同段 LBA; HCA / 滑动窗口本身已一段)	几乎免费（仅分配器小改）	CSA 层 IOPS 省 50 %	必做
v1.5 / v2	Level 2 (per-(prefix, layer) append-stream)	中等：写指针管理 + 引用回收分段 + 共享路径合并	整 prefix 召回 IOPS 再省 ~1 000×, SSU 顺序化	性能优化阶段引入

Config 钩子

为后续平滑切换，kv.physical_layout 中预留拼接级别字段：

```
kv:
  physical_layout:
    stripe_unit: "(prefix_id, layer_id, chunk_id)"      # v1 默认
    stripe_concat_level: 1                             # 1 = Level 1 (默认); 2 = Level 2 流式拼接
    appendstream_chunks_per_segment: 0                 # Level 2 时 > 0: 每段最大 chunk 数 (控制段大小上限)
```

v1 阶段 stripe_concat_level = 1 生效，分配器 / Meta server 实现路径与朴素方案差异极小；v1.5 切到 2 时由 Meta server 走另一套 LBA 段管理。

3.5 Prefix 索引粒度：固定 chunk（不按 token）

3.5.1 三个角度均指向 chunk

角度	token-by-token	固定 chunk（如 64）
索引规模	1 节点 / token; 128 K prompt 库 1 万条 → 1.28 G 节点	1 节点 / 64 token; 同样规模 → 20 M 节点 , 约 2 GiB 内存 , 鲲鹏 256 GiB 内存可承载
IO 单元 / 读放大	单层每 token 1 152 B, 远小于一次 LBA 读 (4 KiB) → 写读放大 ≥ 3.5×	chunk=64 → 单层 chunk 72 KiB = 18 LBA, 对齐无放大

角度	token-by-token	固定 chunk（如 64）
匹配开销	每 token 一次 trie hop；前缀 8 K → 8 K 次哈希	每 chunk 一次；8 K prefix → 128 次
prefix 共享率	极细，但绝大多数共享在前几百 token； 细粒度的边际收益小	粗一些，但实践上人写 prompt 的共享多在 ≥ 64 token 段 (system prompt、模板、文档片段)

3.5.2 推荐参数（V4 Hybrid Attention 下重新校准）

V4 chunk_tokens 必须满足三个**整除约束**（否则 chunk 与压缩组 / 滑动窗口边界错位，读写极复杂）：

```
chunk_tokens % m_csa = 0          # m_csa = 4
chunk_tokens % m_hca = 0          # m_hca = 128
chunk_tokens >= W                 # W = 128, 避免 chunk 内不到一个滑动窗口长度
```

最小满足者 = lcm(4, 128) = **128**；推荐 **chunk_tokens = 512**（128 的 4 倍，HCA 层每 chunk 4 个 entry，避免 HCA chunk 退化到 1 entry → LBA 严重 padding）。

chunk_tokens	CSA 层每 chunk entries	HCA 层每 chunk entries	CSA chunk 字节 (V4 Pro)	HCA chunk 字节 (V4 Pro)	1 M prefix 索引节点数
128	32	1	36 KiB	1.1 KiB（只 1 LBA）	8 192
512（推荐）	128	4	144 KiB	4.5 KiB	2 048
1 024	256	8	288 KiB	9 KiB	1 024
4 096	1 024	32	1.125 MiB	36 KiB	256

为什么 V3 时代的 **chunk=64** 不再适用 V4：HCA 层 m=128 强制 chunk_tokens >= 128；chunk=64 在 HCA 层连一个 entry 都凑不齐。

V4 Pro / V4 Flash 取同一 **chunk_tokens=512**，便于 KV Meta server 用统一逻辑服务两类模型。

3.5.3 chunk 与 prefill 的天然契合

prefill 计算本身按 micro-batch / token-block 推进；chunk_size 选 64 时，每完成 64 token 即可触发一次"chunk emit → 写 SSU → 上报 meta"，与计算 / IO 的流水线天然吻合。

3.5.4 Prefix cache 键的语义：token id 序列（不是 plain text，不是 latent）

cache 命中的判定基础是"prefix 完全相同 → KV 完全相同"。这要求 cache key 满足两个性质：(1) **确定性**（同一前缀两次 tokenize 必产同一 key）和 (2) **精确性**（key 相同 ⇒ KV 字节相同）。系统中只有 **token id 序列** 同时满足这两条。

系统采用：**tokenize 后的 u32 token id 序列**，按 **chunk_tokens** 切片再哈希

```
prompt(plain text)                                ← 用户输入
|
▼ F.tokenize() （v1 在前端 service_access, §5.8）
token_seq[N] of u32 ids
|
▼ 按 chunk_tokens=512 切片 (§3.5.2)
chunks[ [N / 512] ], 每片 = u32[512] (最后一片可短)
|
▼ hash64(chunk_bytes) (推荐 xxhash3 / wyhash)
chunk_hashes[ [N / 512] ]
|
▼ 与 model_id 拼成 radix tree 路径 (见 §3.6.2 model_id)
cache key = (model_id, chunk_hashes[0], chunk_hashes[1], ...)
```

chunk_hash 的输入是纯 **token id 字节流** ($u32 \times 512 = 2 \text{ KiB}$)，完全不涉及 **embedding / hidden state / 任何 latent**。

不在 **plain text** 上匹配的原因

问题	例子 / 影响
文本相同 ≠ token 相同	"Hello,world" vs "Hello, world" 多/少一个空格 → token id 序列不同；KV 真的不同； plain text 比较反而会误命中
Unicode 标准化差异	NFC vs NFD、全角/半角、零宽字符、混合大小写 → 字节级文本比较易脆弱
chat template / system prompt wrap	[user] hello [/user] 模板加完才是模型输入；plain text 比较看不到模板
不能反映模型真正"看到"什么	KV 是模型在 token 上算出的，cache 必须按 token 索引

不在 **latent space (embedding / hidden state)** 上匹配的原因

致命问题	详细
KV 对 token 精确依赖	第 i 层 $KV = f(\text{token_id_i}, \text{position_i}, \text{prev_layer_state})$ 的精确函数。 latent 近邻 ≠ KV 近邻 ；用近邻 latent 的 KV 喂给当前 token 会得到错误注意力分布， 模型质量直接崩盘
位置编码敏感	同一 token 在 position 5 与 position 6 的 KV 完全不同 (RoPE 对位置敏感)；latent 难以精确表达"在哪个位置"
匹配从精确退化为近似	latent 高维 → 只能 ANN (HNSW / IVF) + 相似度阈值；引入 假阳性且不可重复
算 latent 本身需要 forward	拿 latent 必须先 prefill；这就回到了"算 KV 才能 cache KV" 的死循环 —— 算到 latent 就已经算到 KV，cache 也不必要了

行业现状对齐

vLLM、SGLang、LMCache、Mooncake 等开源 / 商用服务系统均采用 **token id 序列**作为 prefix cache key，没有任何一家用 **latent**。这不是工程偷懒，是 KV 语义的硬约束。本系统与之对齐。

v1.5+ 的兼容性预留

chunk_hash 的输入字节是 `model_id || u32 token id × 512`；token id schema 由 tokenizer 决定。**升级 tokenizer = 升级 model_id** (旧 cache 自动作废，无错误命中)。

- 同 model 不同微调 (fine-tune) 共用同一 tokenizer 但训练后的 KV 物理字节不同 → 仍需用不同 `model_id` 隔离。

3.6 KV Meta Server 设计

3.6.1 角色

- 运行：1 台鲲鹏 CPU 服务器 (§1.6 三档场景中 kv_meta.instances = 1)
- 职责：
 - i. 维护全局 prefix radix tree (key = token chunk hash 序列)
 - ii. 维护 SSU LBA 分配池 (每 SSU 一个 free list)
 - iii. 响应 prefix match 查询 (最长前缀匹配)
 - iv. 响应 chunk 写入请求 (分配 (ssu_id, lba); 事务性插入 trie)
 - v. 维护 chunk 引用计数; 触发回收
- 接入网络：
 - **SO**: hot-path RPC (uRPC over UB Urma) —— PrefixMatch / AllocateChunk / CommitChunk / ReleaseRefs / Heartbeat
 - **DCN**: 运维与管理面 (gRPC 或 JSON/HTTP) —— 配置下发、Prometheus 指标拉取、健康检查、日志收集; 外部接入与 §1.7 POSIX FS (snapshot 落盘)

3.6.2 数据结构 (prefix radix tree)

多模型隔离: radix tree 第一层按 model_id 分支; 同一 chunk_hash 在不同 model_id 下落在不同子树, 避免跨模型错误命中 (详见 §3.5.4)。

```
RadixForest {
    per_model: Map<model_id, RadixNode>      // 每模型一棵子树
}

RadixNode {
    edge_label:    u64 chunk_hash[]          // 该边对应的 chunk 序列哈希
    chunk_records: ChunkRecord[]             // 该路径上每个 chunk 的元信息
    children:      Map<u64, RadixNode>       // 下一级 chunk_hash → 子节点
    refcount:      u32
}

ChunkRecord {
    chunk_id:      u64                      // 该 chunk 的全局唯一 id
    model_id:      u32                      // 所属模型 (V4 Pro / V4 Flash / 微调变体均独立)
    layer_blocks:  [SSU_id, lba_start, lba_count][num_layers]
                                              // 每层在哪个 SSU、哪段 LBA
    create_ts:     u64
}
```

V4 ChunkRecord 比 V3 复杂: 每层除了 entry 块, CSA 层还有 indexer 块; 此外每 prefix 还有滑动窗口块。

```
ChunkRecord {
    chunk_id:      u64
    model_id:      u32                // 多模型命名空间隔离
    layer_blocks:  [LayerBlock][num_layers]  // 每层一条
}

LayerBlock {
    layer_type:    u8                // 16 B + 可选 8 B = 16/24 B
    ssu_id:        u8                // CSA | HCA
    entry_lba_start: u32
    entry_lba_count: u16
    // 仅 CSA 层有以下两个字段:
    indexer_lba_start: u32          // FP4 indexer 块起点 (同 SSU)
    indexer_lba_count: u8          // 通常 1 LBA
}

PrefixWindow {
    layer_window_blocks: [SSU_id, lba_start, lba_count][num_layers]
}
```

ChunkRecord 大小估算 (V4 chunk_tokens=512; 含 chunk_id + model_id 头部 16 B):

模型	num_layers	每层 LayerBlock 字节	ChunkRecord 大小	1 M prompt (chunk_tokens=512 → 2 048 chunk)
V4 Pro	80	~16 B (HCA 层) + ~24 B (CSA 层), 按 50/50 平均 ~20 B	16 + 80 × 20 = 1 616 B / chunk	2 048 × 1 616 ≈ 3.16 MiB metadata
V4 Flash	32	同上	16 + 32 × 20 = 656 B / chunk	2 048 × 656 ≈ 1.28 MiB metadata

注: V4 因 chunk_tokens 从 64 升到 512 (×8), 同等 prompt 长度下 chunk 数缩小 8×; 尽管单 ChunkRecord 略大, **总 metadata 反而比 V3-style 小**。

3.6.3 查询流程 (prefix match)

何时被触发: 服务接入服务器 (§1.6 service_access) 收到入口 HTTP/gRPC 请求并完成 tokenization 之后; 调度器决定 worker 之前。每条请求一次 RPC。

算法 (compressed radix tree 上的最长前缀匹配):

```
input: token_seq[N]
1. 把 token_seq 按 kv_chunk_tokens=512 切成 chunk_hashes[ ceil(N/512) ]
2. 从 radix root 开始, 按 chunk_hashes 顺序逐级匹配 children
   - 边压缩匹配 (边 label 是一段 chunk_hash 序列): 到第一个不匹配的 hash 为止
3. 在最深匹配节点处返回:
   - matched_chunks: ChunkRecord[] (hit 部分, 按 chunk_id 顺序)
   - unmatched_tail: token_seq 中未命中的尾部 (送 prefill 计算)
4. 对 matched_chunks 中每个 ChunkRecord 的 refcount += 1 (绑定到本请求 ID)
5. 返回 (matched_chunks, unmatched_tail) 给调度器
```

复杂度: O(N / chunk_tokens) 次 hash 比较 (每层 trie 节点常数时间)。1 M context、512 chunk_tokens 下最深 ~2 048 跳; 指针追逐主导 → 实测 < 几百 μs。

控制路径 vs 数据路径:

阶段	走的网络	谁发起	内容
① 查询 RPC	SO (uRPC over UB Urma)	service_access CPU → meta server CPU	token_seq 或 prompt hash list
② 返回 ChunkRecord list	SO (uRPC 回包)	meta server → service_access	每 chunk 的 (layer, ssu_id, lba_start, lba_count) 列表
③ 调度器把 ChunkRecord list 注入 worker	SO (uRPC 控制面)	service_access → 选定的 prefill/decode 节点 host CPU	同上 list (按目的 NPU 分发, 可仅给一个 NPU 因 DP-attention)
④ NPU 直接读 SSU 拉 KV bytes	SO 网络 (UBG 协议)	NPU 自身 (DMA 引擎 / SO RDMA)	按 ChunkRecord 中的 (ssu_id, lba_start, lba_count) 直接发起 SO read

关键设计点：Meta server 只负责返回"**chunk record 列表**", 不参与任何 KV 字节的搬运。字节路径完全是 **NPU ↔ SSU 直连** (经 SO 交换), CPU 完全不在数据路径上。这是 §1.3 / §1.4 的 SO 网络 any-to-any 性质带来的关键收益。

3.6.4 写入流程 (prefill / decode 完成新 chunk 后)

两个触发源 (v1 都支持, 行为一致):

触发源	何时产生新 chunk	何时调用 meta server
prefill	把不命中的尾部 prompt 计算成 KV, 每填满 chunk_tokens=512 个 token 即生成一个新 chunk	每完成一个 chunk 立即提交 (流水化)
decode	自回归生成累积到 chunk_tokens 边界	同上

两源使用 **完全相同的 RPC 接口** (见下); meta server 不区分 chunk 来自 prefill 还是 decode。

控制路径 vs 数据路径:

步骤	谁发起	经过	网络
① AllocateChunk RPC (请求 LBA)	NPU 所属 PC16 服务器的本机 CPU (host)	接收 NPU 通过 NPU↔CPU 8×400 Gbps UB (§1.2) 的 doorbell / mailbox 触发 host	SO uRPC (over UB Urma) → meta server
② 返回 LayerBlock[num_layers]	meta server	host CPU 把分配结果 DMA 推回 NPU 的 mailbox	SO uRPC → host → NPU
③ 写 KV 字节到 SSU	NPU 自身 (DMA / SO Urma)	不经 CPU, 按 LayerBlock 直接写入对应 SSU 的 LBA 段	SO (Urma write)
④ NPU 写完 → 触发 host 完成中断	NPU → host CPU (NPU↔CPU UB)	-	服务器内部 UB
⑤ CommitChunk RPC (注册 ChunkRecord)	host CPU	把 chunk_hash + LayerBlock[] 提交	SO uRPC → meta server
⑥ meta server 原子插入 trie; 该 chunk 对后续 query 可见	meta server	-	-

为什么把 RPC 放给 host CPU 而不是 NPU 直接发:

- NPU 上虽有 SO/UB 接入，但不承担完整的 **uRPC 调用栈**（uRPC 的 stub 生成、序列化、超时重传、错误码处理等放在 host CPU 上）；NPU 只做数据面 Urma read/write
- 元数据请求频率 = 每 chunk 一次（chunk_tokens=512 → 每 ~512 token 一次），相比 token 级算力流量是噪声级
- host CPU 发起便于：批量提交（多个 chunks 合并为一次 RPC）、错误重试、与 PC16 本地的请求簿记一致

为什么 SSU 字节写不经 CPU：

- §1.3 SO 网络是 NPU/CPU/SSU any-to-any，NPU 直发 SSU 是 1 跳
- 若经 CPU 转发：HBM → CPU → SSU 需占用 NPU↔CPU UB（3.2 Tbps / NUMA 共享）+ CPU 网卡 SO（400 Gbps），两次跳转浪费带宽
- KV 字节流量大（CSA chunk 148 KiB / chunk × 22 CSA 层 / chunk_tokens=512 → 每 token 写入 ~6 KiB），不能让 CPU 进数据路径

RPC 接口（uRPC over UB Urma，承载在 SO；host ↔ meta server）：

```
// uRPC 服务定义（IDL 兼容 protobuf；底层走 UB Urma 而非 HTTP/2）
service KVMeta {
    // 查询（service_access 调用）
    rpc PrefixMatch(PrefixMatchReq) returns (PrefixMatchResp);
    rpc ReleaseRefs(ReleaseRefsReq) returns (Empty); // 请求结束后归还引用

    // 写入（host CPU 代 NPU 调用，prefill 与 decode 共用）
    rpc AllocateChunk(AllocateReq) returns (AllocateResp);
        // AllocateReq: prefix_id, chunk_id, layer_types[num_layers]
        // AllocateResp: LayerBlock[num_layers] // (ssu_id, lba_start, lba_count) per layer
    rpc CommitChunk(CommitReq) returns (Empty);
        // CommitReq: prefix_id, chunk_id, chunk_hash, LayerBlock[num_layers]
    rpc AllocateWindow(WindowReq) returns (WindowResp); // prefix 收尾时一次
    rpc CommitWindow(CommitWindowReq) returns (Empty);

    // 维护
    rpc Heartbeat(HostState) returns (CapacityHint); // 容量水位回传给 host
}
```

伪代码（host CPU 视角）：

```
on_npu_chunk_ready(prefix_id, chunk_id, layer_types[]):
    1. resp = meta.AllocateChunk(prefix_id, chunk_id, layer_types)
    2. 把 resp.LayerBlock[] DMA 推到 NPU mailbox
    3. NPU 按 LayerBlock 通过 SO 直接写各 SSU
    4. 等 NPU 完成中断
    5. meta.CommitChunk(prefix_id, chunk_id, chunk_hash, LayerBlock[])
    6. （可选）累积多个 commit 合并为一次 RPC（v1.5 优化）
```

滑动窗口在 prefix 收尾时单独走一次 AllocateWindow / CommitWindow

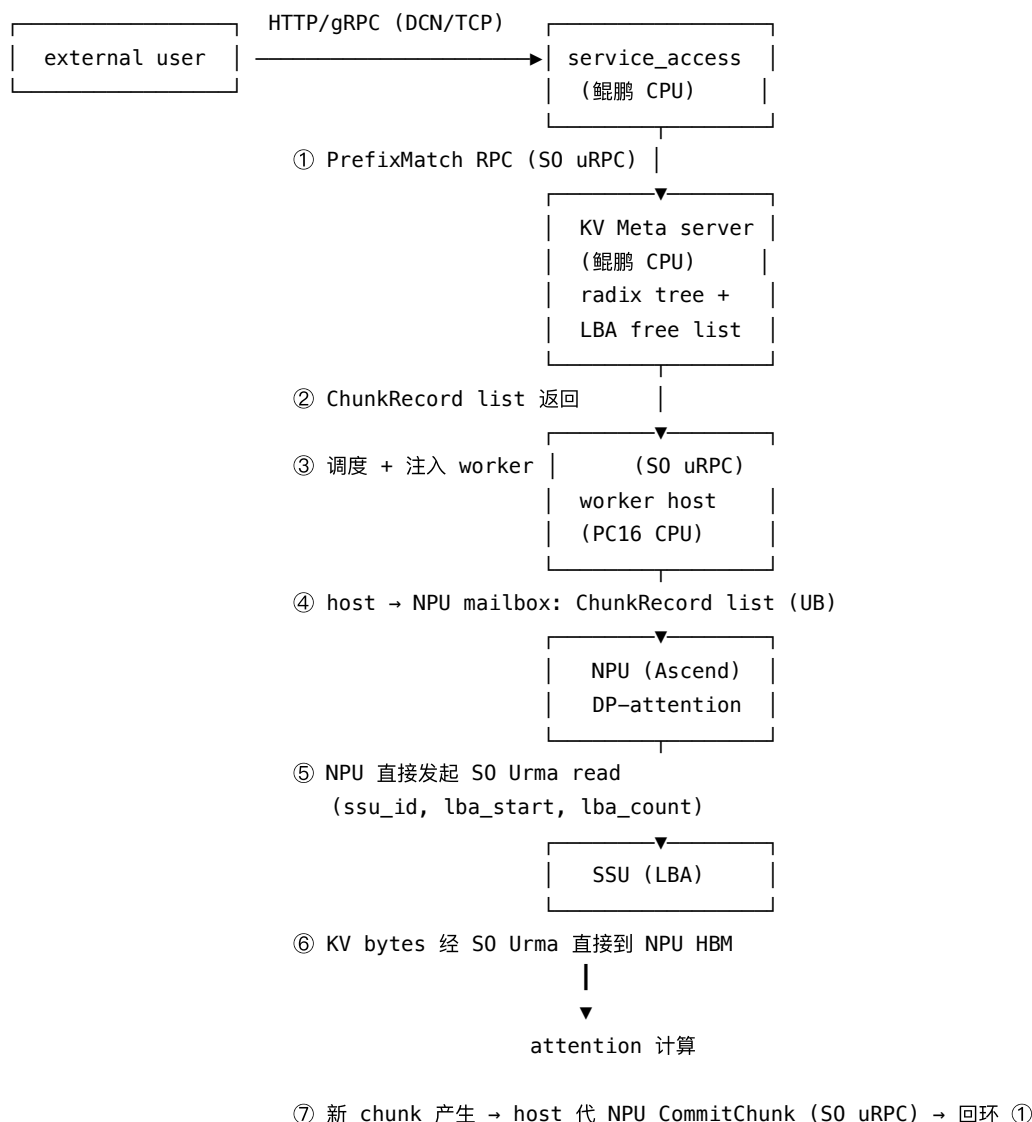
对后续可见性：CommitChunk 一旦返回，新 chunk 立即对其他请求的 PrefixMatch 可见（trie 写完即可读，§3.6.8 的 RCU 语义）。

3.6.5 容量与扩展

- 内存占用估算（V4 chunk_tokens=512，1 M 个独立 prefix × 平均 8 K token = 1 M × 16 chunk = **16 M ChunkRecord**）：
 - **V4 Pro** (~1 600 B / chunk)：16 M × 1 600 B ≈ **25 GiB**
 - **V4 Flash** (~640 B / chunk)：16 M × 640 B ≈ **10 GiB**
 - 加 trie 结构开销（每节点 ~200 B）+ free list → Pro 总占用约 **35–45 GiB**；Flash 约 **15–20 GiB**

- 鲲鹏 256 GiB 单实例 v1 充分余量；相比 V3-MLA 方案的 ~150 GiB 估算下降 ~5x (chunk_tokens 从 64 升到 512 → chunk 数 / 8；ChunkRecord 略大但总量大幅缩小)
- 进一步扩展 (多模型共享、上亿 prefix) 走 §3.7 的多 meta server 路线
- **持久化**：v1 把 trie + free list 周期性 **snapshot 到 §1.7 的 DCN POSIX FS** (非 SSU)，作为崩溃恢复的备份；运行时只在内存 (详见 §3.6.7)
- **后续扩展**：分片到多 meta server 实例 (按 prefix_id 哈希)，高可用 raft，热复制

3.6.6 控制路径 vs 数据路径汇总 (端到端示意)



关键不变量：

- 元数据热路径 = SO 上的 **uRPC over UB Urma**，host CPU ↔ meta server，**永远不经 NPU** (NPU 不承担 uRPC stack)
- 数据路径 = SO 上的 **Urma read/write**，NPU ↔ SSU 直连，**永远不经 CPU**
- SO 上控制面 uRPC (短包、低频) 与数据面 Urma 大块 read/write (高带宽) 通过 Urma 队列对 (QP) 天然隔离调度，互不阻塞
- 协议-网络映射 (与 §1.3 一致)：
 - 走 SO/SU 的 RPC = uRPC (非 gRPC)；用于 PrefixMatch / AllocateChunk / CommitChunk / Dispatch 等 hot-path
 - 走 DCN/RoCE 的 RPC = gRPC 或 JSON/HTTP；用于外部 ingress、Prometheus 指标拉取、运维 / 部署 / 健康检查 (非 hot-path)

3.6.7 持久化与崩溃恢复

v1 策略：内存优先 + 周期 snapshot 到 POSIX FS

项	v1 取值
运行时存储	全内存（鲲鹏 256 GiB DRAM）
持久化目标	§1.7 DCN POSIX FS（NFS / 3FS / S3-like）
Snapshot 路径	posix_fs:///kvmeta/snapshots/<ts>.bin
Snapshot 频率	60 秒 一次（可配 kv_meta.snapshot_interval_s）
Snapshot 内容	trie 全树 + 每 SSU 的 free list + ChunkRecord LRU 链表
Snapshot 方式	增量 fork + COW（v1 用简单 fork+memcpy；v1.5 上 COW）
WAL	v1 不做；以 60 s 滚动 snapshot 为粒度
崩溃恢复	从最新 snapshot reload；snapshot 之间提交的 chunk 全丢

崩溃语义（与 §3.7 "KV 是可丢失缓存"姿态一致）：

- 丢失的 ChunkRecord → 该前缀对后续请求表现为 PrefixMatch miss → prefill 重算
- SSU 上的字节虽然还在但无 metadata 指向 → 视为垃圾，由后台 GC 扫描归还（详见 §3.6.9）
- 服务连续性：恢复期 service_access 暂停下发新请求；恢复完成（< 几十秒）即可继续。**不做高可用主备**（v1 留白，§3.7）。

snapshot 大小估算（接 §3.6.5）：V4 Pro ~40 GiB / V4 Flash ~18 GiB；POSIX FS 写带宽假设 10 GiB/s → snapshot 写完 ~4 s（Pro） / ~2 s（Flash）。snapshot 期间的写请求短暂 stall（CommitChunk 卡 ~几百 ms）—— v1 可接受；v1.5 引入 COW 即可消除。

3.6.8 并发控制与原子性

v1：单写者 + RCU 读

操作	锁/同步策略
PrefixMatch（读）	无锁；RCU snapshot 读 trie。trie 节点带 epoch；写者切换 epoch 后旧节点延迟回收
AllocateChunk（分配 LBA）	每 SSU 一把 spinlock 保护其 free list；多 SSU 间无锁
CommitChunk（写 trie）	按 chunk_hash 路径加锁（path-lock），不同 prefix 的并发 commit 互不阻塞
ReleaseRefs（refcount 减）	原子计数（atomic_fetch_sub）
eviction 后台线程	单线程；与 commit 协作走 path-lock

原子性边界：

- AllocateChunk 与 CommitChunk 之间是 **两阶段提交**：
 - Phase 1 (Allocate)：LBA 已从 free list 扣除，但 trie 中尚无 ChunkRecord → 资源已占用、对查询不可见
 - Phase 2 (Commit)：trie 中插入 ChunkRecord → 对查询可见
- 若 NPU 写 SSU 失败（SO 链路错误等）：host CPU 收到 NPU 错误 → 调用 AbortChunk(prefix_id, chunk_id) 把 LBA 归还 free list，不调 Commit
- 若 Allocate 后 host 崩溃 / 超时未 Commit：meta server 设 30 s timeout，自动 abort 所有 in-flight Allocate（被孤立的 LBA 由 §3.6.9 的 GC 收回）

3.6.9 SSU 空间生命周期：分配 / 释放 / Recycle

每 SSU 独立维护一棵 **free extent tree**（按 lba_start 排序的红黑树 + 按 size 索引的多级 free list）。

分配（AllocateChunk 内部）

```
allocate_lba(ssu_id, want_lba_count):
  1. ssu = ssu_pool[ssu_id]
  2. with ssu.lock:
    # 优先精确尺寸 freelist
    if exact_match in size_freelist[want_lba_count]:
      return pop()
    # 否则 best-fit on extent tree
    extent = extent_tree.find_smallest_ge(want_lba_count)
    if extent.size > want_lba_count:
      split → 一段返还 freelist, 一段返回
    return (lba_start, want_lba_count)
```

- 选择 SSU 由 §3.4 的 stripe hash 决定，**不是**这里挑（这里只在选定的 SSU 内分配 LBA range）
- 算法：**多级 free list (buddy-like) + best-fit on extent tree**，对 §3.4.5 的 4 种典型尺寸（37 / 29 / 2 / 1 LBA）做尺寸分类，避免常态 split / merge

释放（eviction 或 abort 时）

```
free_lba(ssu_id, lba_start, lba_count):
  1. with ssu.lock:
    extent_tree.insert(lba_start, lba_count)
    # 与左右相邻空闲段合并
    coalesce_with_neighbors()
    重新挂入对应 size_freelist
```

回收来源（共三类）

回收来源	触发条件	处理路径
正常 eviction	LRU 选中 cold ChunkRecord 且 refcount=0	meta server 调 free_lba + 从 trie 移除
Abort (写失败 / 超时)	AllocateChunk 后 30 s 内未 Commit	meta server timer 调 free_lba
孤立段 GC (崩溃恢复后)	snapshot 之间分配但未 commit 的段	启动时全 SSU 扫描：总容量 - sum(已知 LayerBlock) 即孤立段，归 free list

是否 erase SSU 上的字节

不擦（懒回收）：free 只是 metadata 操作；SSU 上原 bytes 保留直到下次写覆盖。原因：

- KV 不含敏感持久化数据（每次请求结束即可视为可丢）
- erase 会显著拖慢释放速度
- 安全性留待企业版选项（kv.secure_erase: false）

碎片化

V4 chunk 尺寸离散到 4 类（37 / 29 / 2 / 1 LBA），分类 free list 几乎消灭碎片；长期运行的零散 split 由后台 compaction 在低水位时合并（v1.5+, v1 跳过）。

3.6.10 Eviction 策略

触发条件：

高水位 high_watermark = 85% （per-SSU 容量）
低水位 low_watermark = 75%

某 SSU 容量超过 high_watermark 时启动 eviction，回收到 low_watermark 之下停止。

算法选型：

算法	v1 选	理由
LRU（按 last_access_ts 排序）	是	实现最简，对 prompt 复用模式（system prompt 反复命中）已经足够
LFU	否	维护命中频次成本高；不适合 v1
W-TinyLFU	否（v1.5 候选）	高级方案，但 v1 不引入
TTL（绝对时间淘汰）	否	与 LRU 正交，可作为兜底（v1.5）

ChunkRecord 增字段（与 §3.6.2 LayerBlock 互补）：

```
ChunkRecord {
    ...                // §3.6.2 字段
    last_access_ts: u64    // PrefixMatch 命中时更新
    refcount:          u32  // inflight 引用数 (>0 不可 evict)
    pinned:            bool // 显式钉住 (v1.5: system prompt 等)
}
```

算法（每 SSU 一条 LRU 链表，按 last_access_ts 排序）：

```
evict_until_low_watermark(ssu_id):
    while ssu.usage > low_watermark:
        cr = lru_list[ssu_id].head    # 最 cold
        while cr.refcount > 0 or cr.pinned:
            cr = cr.next                # 跳过仍被引用 / pinned
            if cr is None: return       # 全在用，等下个周期
        # 实际淘汰
        for layer_block in cr.layer_blocks:
            free_lba(layer_block.ssu_id, layer_block.lba_start, layer_block.lba_count)
        # 注意: cr 在多个 SSU 上有 layer_block, 仅释放本 SSU 上的会留半残;
        # v1 选择整 ChunkRecord 一并淘汰（即跨多 SSU 同时释放）以保索引一致性
        radix_tree.remove(cr)          # path-lock 保护
        lru_list.remove_each_ssu(cr)
```

关键边界处理：

- **跨 SSU 一致性**：一个 ChunkRecord 的 LayerBlock 散落在 ~num_layers 个 SSU 上。**eviction 必须以 ChunkRecord 为整体单位**（要么全淘要么全留），不能只释一部分 layer 否则索引半残。所以单一 SSU 容量压力会触发跨 SSU 的同步回收。
- **trie 节点的级联清理**：ChunkRecord 移除后，若其所在 trie 节点 children 为空且自身无 ChunkRecord → 上溯合并，保持 trie 紧凑。
- **滑动窗口的回收**：与 prefix 整体绑定（每 prefix 一份），prefix 整体被淘汰时一次性释放。
- **公平性**：v1 简单 LRU；多租户场景的反淘汰可在 v1.5 引入按 tenant 配额。

eviction 行为可观测项（指标输出到监控）：

指标	含义
evictions_per_sec	触发频率
bytes_reclaimed_per_sec	速率
eviction_skip_refcount	因 refcount>0 跳过的次数
eviction_skip_pinned	因 pinned 跳过的次数
prefix_match_hit_rate	间接反映 eviction 是否过激
ssu_usage_per_ssu	每 SSU 容量水位

v1.5 / v2 升级路径：

项	改进
LRU → W-TinyLFU	抗扫描污染（一次性长 prompt 不 evict 频用 prompt）
加 TTL 兜底	防 leak
多 tenant 配额	公平调度
主动 prefetch	高频 system prompt 自动 pin 在多 SSU 副本（与多副本 §3.7 联动）

Config 钩子

```
kv_meta:
  eviction:
    algorithm:      lru          # v1 = lru; v1.5 可选 w_tinylfu
    high_watermark_pct: 85
    low_watermark_pct: 75
    refcount_timeout_s: 300      # refcount > 0 超过此值视为泄漏，强制清零
    allocate_timeout_s: 30       # AllocateChunk 后未 Commit 的超时
    secure_erase:    false      # 释放时是否擦写 SSU bytes
  persistence:
    snapshot_target:  "posix_fs:///kvmeta/snapshots/"
    snapshot_interval_s: 60
    snapshot_mode:     fork_copy  # v1.5 可选 cow
    wal_enable:        false      # v1 关; v1.5 开
```

3.7 v1 暂不实现的内容（明确留白）

项	v1	后续
KV 块副本	replication_factor = 1，单点	多副本（2 / 3）+ stripe-erasure
SSU 故障恢复	不做；故障 SSU 上的 KV 视为丢失，下一次 prefix 命中失败时按 miss 重新计算	副本切换、后台 rebuild
Meta server 高可用	1 实例；崩溃时整库失效	主备 / raft 多副本；分片
Trie 持久化频率	周期 snapshot 到 POSIX FS	WAL + 增量 snapshot

项	v1	后续
跨数据中心 KV 镜像	不做	异步复制

设计姿态：v1 把 KV 视为**可丢失的可重建缓存**。任何丢失都退化为 prefix miss，再次按 prefill 路径计算 —— 正确性不依赖 KV 持久性。

3.8 配置参数小结（与 §2 一致：全部由配置驱动）

```
# ----- V4 Hybrid Attention 通用参数 (Pro / Flash 共用) -----
attention:
  type:          hybrid_v4          # CSA + HCA + sliding window
  m_csa:         4                  # CSA 序列压缩比
  m_hca:         128                # HCA 序列压缩比
  sliding_window: 128              # W
  indexer_dtype:  fp4               # Lightning Indexer 精度
  layer_schedule: "csa,hca,csa,hca,..." # 各层类型, 由模型决定

# ----- V4 Pro 模型相关 -----
model:
  name:          deepseek-v4-pro
  num_layers:    80                # 预估值, 待官方
  csa_entry_bytes: 1152            # 预估值; 待官方规格
  hca_entry_bytes: 1152            # 预估值
  window_token_bytes: 1152        # 预估值
  indexer_per_entry: 32            # FP4, head_dim 量级 × 0.5

# ----- V4 Flash 模型相关 -----
model:
  name:          deepseek-v4-flash
  num_layers:    32                # 预估值
  csa_entry_bytes: 896
  hca_entry_bytes: 896
  window_token_bytes: 896
  indexer_per_entry: 32

# ----- KV cache 物理布局 (与模型 / 硬件场景均无关) -----
kv_cache:
  chunk_tokens:  512              # 必须满足 % m_hca == 0 && >= sliding_window
  layout:        per_layer_chunk # §3.4 方案 (c)
  csa_chunk_bytes_per_layer:      "(chunk_tokens / m_csa) * csa_entry_bytes" # 派生
  hca_chunk_bytes_per_layer:      "(chunk_tokens / m_hca) * hca_entry_bytes"
  indexer_chunk_bytes_per_layer:  "(chunk_tokens / m_csa) * indexer_per_entry" # 仅 CSA 层
  stripe_hash:    "hash(prefix_id, layer_id, chunk_id) % n_ssu"
  indexer_co_locate: true         # CSA entry chunk 与 indexer chunk 同 SSU
  replication_factor: 1          # §3.7
  ssu_lba_size:    4096

kv_meta:
  index:          radix_tree      # §3.6.2
  # ---- 持久化 (§3.6.7) ----
  persistence:
    snapshot_target: "posix_fs:///kvmeta/snapshots/" # §1.7 POSIX FS
    snapshot_interval_s: 60
    snapshot_mode:      fork_copy      # v1.5 可选 cow
    wal_enable:         false          # v1 关; v1.5 开
  # ---- 并发控制 (§3.6.8) ----
  concurrency:
    read_model:        rcu          # 查询无锁; 写者切换 epoch
    per_ssu_alloc_lock: spinlock     # free list 保护
    commit_lock:        path_lock    # 按 chunk_hash 路径加锁
    allocate_timeout_s: 30           # AllocateChunk 后未 Commit 的超时 (自动 abort)
  # ---- Eviction (§3.6.10) ----
  eviction:
```

```
algorithm:          lru                                # v1 = lru; v1.5 可选 w_tinylfu
high_watermark_pct: 85
low_watermark_pct:  75
refcount_timeout_s: 300                                # refcount > 0 超过此值视为泄漏, 强制清零
secure_erase:       false                              # 释放时是否擦写 SSU bytes
# ---- RPC (§3.6.4 接口) ----
rpc:
# 数据/计算 hot-path: PrefixMatch / AllocateChunk / CommitChunk / Dispatch
hot_path:
  transport:        urpc                                # uRPC over UB Urma (S0/SU 上的 RPC 一律 uRPC, 不走 gRPC)
  bind_network:      so                                  # 与 KV 数据面共 S0 物理网, 协议上 QP 隔离
  qp_count:          16
  max_inflight:      65536
# 运维 / 监控 / 健康检查 / 配置下发: 与 hot-path 协议解耦, 走 DCN 标准协议
ops:
  transport:         grpc                                # gRPC 或 JSON/HTTP; DCN 上的 RPC 一律 gRPC (§1.3 协议-网络映射)
  bind_network:       dcn
  ports:              { metrics: 9090, health: 8080, admin: 9000 }
```

切换模型：只需替换 `model`：块 (`num_layers`、各 `entry_bytes`、`layer_schedule`)；`attention`：与 `kv_cache`：的字段 (`chunk_tokens`、`layout`、`stripe_hash`、`replication_factor`、SSU LBA) 在 V4 Pro / V4 Flash 与三档硬件场景间保持一致。

切换硬件场景 (§1.6 A/B/C)： `kv_cache`：全字段都不变；只需调整与 §1.6 / §2 / §4 相关的算力侧参数。

4. 并行化策略：FFN 用最大 EP，Attention 默认 DP

本章面向 **V4 Pro / V4 Flash** (Hybrid Attention + DeepSeekMoE 体系，参见 §3.3.1)，分析 `prefill` / `decode worker` 的并行化策略。

4.1 顶层选择

模块	并行模式	跨 §1.6 三档的取值	简短理由
FFN (MoE 专家路由)	EP (Expert Parallel)	A: EP16 / B: EP64 / C: EP128	每档取硬件能支持的最大 EP，把 总 HBM 带宽 拉满 (§4.3)
Attention (CSA / HCA / 滑动窗口)	DP-attention 默认 (TP=1)	A: DP=16 / B: DP=64 / C: DP=128	V4 序列压缩后单层 KV 已极小，单 NPU HBM 容得下且 scan 时间不主导；TP attention 改为 <i>profile-driven</i> 可调参 (§4.4)
同一 NPU 在 attention 阶段是 DP-attention rank；在 FFN 阶段是 EP rank			同一硬件位点切换并行模式 (无需迁移参数)

DP-attention + EP-FFN 在 V3 MLA 与 V4 Hybrid Attention 下都是合理默认，但底层动机不同：V3 是因为 MLA 不可切 head (TP 纯负收益)；V4 是因为序列压缩后 KV 已极小 (TP 变成可选优化而非默认)。

4.2 FFN 的 EP 三档：随场景递进

场景	NPU 总数	推荐 FFN EP	每 rank 持有专家数 (V4 Pro / V4 Flash)
A (1 PC16 prefill / 1 PC16 decode)	16 NPU	EP16	Pro 256/16 = 16 experts/rank ; Flash 64/16 = 4 experts/rank
B (4 PC16 / 4 PC16)	64 NPU	EP64	Pro 256/64 = 4 experts/rank ; Flash 64/64 = 1 expert/rank
C (1 UBL128 / 1 UBL128)	128 NPU	EP128	Pro 256/128 = 2 experts/rank ; Flash 64/128 (< 1) → EP64 fallback (见 §4.3.3)

4.3 为什么 FFN 优选最大 EP

4.3.1 decode 阶段 FFN 的瓶颈：HBM 带宽

decode 一个 token 的 FFN 路径：

- 1. router 给 token 选 top-K 个专家 (K = 8 / Pro, 6 / Flash)
- 2. 把 token 的激活通过 all-to-all 发到这 top-K 个专家所在 rank
- 3. 各 rank 用自己持有的专家权重做 MLP (gate + up + down 三个 matmul)
- 4. 把结果通过 all-to-all 收回原 rank

第 3 步是**主要时延来源**：每个 token 从 HBM 拉读 K 个专家的权重 (约几十至几百 MB / token，与 hidden / moe_intermediate 相关)。它是**纯 memory-bandwidth bound** 的，因为单 token 算力很低。

4.3.2 总 HBM 带宽随 EP 线性扩展

把同一份 MoE 层的权重 stripe 到 N 个 NPU 的 HBM 上：

- 每 NPU 持权重量 = 1 / N
- 每 NPU 单次 decode 需扫的权重量 = 1 / N (因为 token 路由的 K 个专家在哪些 rank 由路由决定，长期均衡时每 rank 期望命中量正比于持有量)
- 每 NPU 扫描时间 ∝ HBM 单卡带宽相同 → **单层 FFN 时延 ∝ 1 / N**
- 整个集群 总 HBM 带宽 = N × per_NPU_HBM_bw → **线性放大**

这就是用户要求"FFN 优选最大 EP"的根本原因：**最大化集群总 HBM 带宽 = 最小化 decode 时延**。

4.3.3 EP 上限：每 rank 至少一个专家

EP 不能任意大。**约束**：每 rank 至少持 1 个专家，否则会出现"空 rank"。

- V4 Pro (256 routed experts)：EP ≤ 256，C 档 EP128 余 2 experts/rank，**有空间**
- V4 Flash (64 routed experts)：EP ≤ 64，**C 档应回落 EP64** (128 NPU 中 64 NPU 走 FFN-EP，另 64 NPU 可走 EP 副本/数据并行；最终具体方案在 v1 实现时定)

4.3.4 EP 的代价 (all-to-all 通信) 与 SO 容量校核

每 token 走两次 all-to-all (dispatch + combine)，通信量正比于 hidden_size × top_K：

- V4 Pro：top_K=8，hidden ≈ 7168 (预估)，dispatch 流量 ≈ 8 × 7168 × 2 B = 112 KiB / token (含元数据 ~150 KiB)
- V4 Flash：top_K=6，hidden ≈ 4096 (预估)，dispatch 流量 ≈ 6 × 4096 × 2 B = 48 KiB / token

decode batch 32 tokens、每层 2 次 all-to-all、Pro 80 层、SU 单 NPU 3.2 Tbps：

- 每秒 token 数上限（粗算）= $3.2 \text{ Tbps} \times N_{\text{NPU}} / (150 \text{ KiB} \times 2 \times 80 \times \text{tokens_per_step})$
- 在 EP128 下，SU 域内 all-to-all 用 SU 交换机 ($3.2 \text{ Tbps} \times 128 \text{ NPU} = 409.6 \text{ Tbps}$ 域内总带宽)，不构成瓶颈

跨 UBL128 时（多 UBL128 部署）才会用 SO 走 all-to-all，那时 SO 单 NPU 400 Gbps 会成为新边界。三档场景中 prefill / decode 各自不跨 UBL128，故 EP all-to-all 留在 SU 域内、带宽充裕。

4.4 Attention 是否还要追加 TP？（V4 Hybrid Attention 下重新分析）

重要前提： V4 改用 Hybrid Attention（CSA + HCA + 滑动窗口 + FP4 Lightning Indexer），不再是 **MLA**。MLA 时代 "TP attention 没用，因为 KV 是 head-shared latent" 的结论 **不能直接套用** 到 V4。本节按 V4 的三种 attention 组件分别分析。

4.4.1 V4 与 MLA 的关键区别

维度	V3 MLA	V4 Hybrid Attention
KV 压缩方向	特征维（kv_lora_rank=512 共享低秩 latent, head-shared）	序列维（每 m 个 token → 1 entry, m=4 / 128）
KV 是否可按 head 切分	否：head-shared，不可切	是（很可能）：compressed entries 仍含 per-head K/V，只是序列短了
KV 总量量级（1 M context）	~70 GiB / prompt	~5–7 GiB / prompt（-90%）
单层 KV scan 主导时延	是（KV 几 GiB / 层）	不再是（HCA 层 KV 已小到几 KiB / 层 / token）
是否需要 sparse selection	否	是（CSA + Lightning Indexer 选 top-k）

4.4.2 按组件分别分析 TP 收益

(a) HCA 层（m=128, dense attention over compressed entries）

- 每层 entries 数 $\approx N / 128$ （1 M 上下文 → ~7 800 entries / 层）
- 每个 entry 是 per-head K/V（V4 假设；待官方确认）
- $\text{attention compute} = \text{compressed_entries} \times \text{num_heads} \times \text{head_dim}$ （标准 MHA 形态，但序列已极短）
- **TP 切 head 的效果：**
 - per-rank KV HBM $\times 1/\text{TP}$ （heads 切开，对应的 K/V 也跟着切）
 - per-rank scan 时间 $\times 1/\text{TP}$ （heads 并行）
 - 收益 **存在**，但因为 HCA 层 KV 已经极小（~几 MiB / 层 / 1 M context），TP 收益的 **绝对** 时延减少很有限
- 代价：attention 输出后多一次 all-reduce（W_O 之前）；TP=8 在 PC16 内 full-mesh 上 ~10 μs 量级

(b) CSA 层（m=4, sparse top-k selection via Lightning Indexer）

- 每层 entries 数 $\approx N / 4$ （1 M 上下文 → 250 K entries / 层）
- 计算路径分两段：(1) Indexer 用 FP4 K' 与 query 打分 → 选 top-k，(2) 在选出的 top-k entries 上做完整 attention
- **TP 切 head 的效果：**
 - Indexer 打分：可按 head 切；每 rank 持 $1/\text{TP}$ 的 indexer K'；top-k 选择需要 **跨 TP rank 一次 reduce**（否则各 rank 选出不同 top-k）
 - 选中 top-k 后的 attention：per-rank 做 $1/\text{TP}$ heads，与 HCA 层同
 - 整体收益与 HCA 层类似，但多一次 **indexer reduce**
- **额外考虑：** top-k 数通常 $\ll$ 总 entries → 即使不 TP，单 rank 也能在很短时间内完成。CSA 的 sparse 设计本身已经把单层 attention 时延砍到很小。

(c) 滑动窗口 (W=128, 最近 token 未压缩)

- 每层 W=128 raw tokens 的标准 MHA / GQA
- KV 占 144 KiB / 层 (小到忽略)
- TP 切 head 是教科书做法; 收益与传统 MHA 完全一致
- 代价: W_O 前一次 all-reduce

4.4.3 综合判断: V4 下 TP attention 是否需要?

情景	推荐
decode (短输出 / 长上下文)	TP=1 (DP-attention 默认)。V4 KV 总量已经压到 5-7 GiB (1 M context), 单 NPU HBM 完全容纳; scan 时延已经不主导; TP 的相对收益小, 复杂度高
prefill (极大 batch / 极长上下文)	TP=2 或 4 可选。当单 rank 的 indexer 打分 + top-k 路径占 prefill 时延 ≥ 20% 时, 开 TP; TP 边界 ≤ 8 (限 PC16 内 8-NPU full-mesh)
HBM 极紧 (少 NPU 部署)	TP=2 或 4 可选。如场景 A (16 NPU), 用小 TP 把 indexer / Q / W_O 权重分担
跨 UBL128 部署	TP 不应跨 UBL128 (all-reduce 走 SO 价格高); TP 限同 PC16 8-NPU 组内

默认推荐 (写入配置):

- 场景 A: tp_attn = 1 (NPU 数本身就少, DP 优先)
- 场景 B: tp_attn = 1 (同上)
- 场景 C: tp_attn = 1 (V4 KV 已极小, 无需 TP; profile 后若 prefill indexer 打分热, 可调到 2 / 4)
- prefill 调优期: 允许 tp_attn ∈ {1, 2, 4, 8} 用 §2 的配置切换实验

4.4.4 与 §3 KV cache 协同 (V4 下重新核对)

- DP-attention + per-(layer, chunk) SSU stripe (§3.4) 的"一份 KV 只加载一次"性质 仍然成立
- 若开 TP attention (TP=k): CSA / HCA entries 必须 复制 ×k (因为按 head 切到 TP rank, 每 rank 仍要看到全 entries 数)
 - 这是因为: 序列维已经被压缩, TP 切 head 不能进一步切序列维
 - 每 prefix 命中时, SSU → NPU 的加载量也 ×k
- 因此 TP attention 在 V4 下仍然 付出 ×k 的 KV 加载代价, 与 V3 MLA 同; 但 收益侧 在 V4 下比 V3 MLA 多了"per-rank scan 真正按 1/k 缩"这一项 (V3 MLA 是 0 收益)

一句话总结: V4 把 TP attention 从"MLA 时代纯负收益"提升为"小幅正收益但默认仍不开"。开不开 TP 在 V4 下变成 profile 驱动的可调参, 而不是 MLA 时代那种 架构层面的禁忌。

4.5 落到配置 (更新 §2.5 三档 YAML)

把 parallel 字段拆开为更细的并行项:

```
# 场景 A
prefill: { scope: pc16,          servers: 1, parallel: { ep: 16,  tp_attn: 1, dp_attn: 16,  sp: 1 } }
decode:  { scope: pc16,          servers: 1, parallel: { ep: 16,  tp_attn: 1, dp_attn: 16,  sp: 1 } }

# 场景 B
prefill: { scope: ubl128_partial, servers: 4, parallel: { ep: 64,  tp_attn: 1, dp_attn: 64,  sp: 1 } }
decode:  { scope: ubl128_partial, servers: 4, parallel: { ep: 64,  tp_attn: 1, dp_attn: 64,  sp: 1 } }

# 场景 C — V4 Pro
prefill: { scope: ubl128_full,    servers: 8, parallel: { ep: 128, tp_attn: 1, dp_attn: 128, sp: 1 } }
decode:  { scope: ubl128_full,    servers: 8, parallel: { ep: 128, tp_attn: 1, dp_attn: 128, sp: 1 } }

# 场景 C — V4 Flash (experts 仅 64, FFN EP 回落到 64)
prefill: { scope: ubl128_full,    servers: 8, parallel: { ep: 64,  tp_attn: 1, dp_attn: 128, sp: 1 } }
decode:  { scope: ubl128_full,    servers: 8, parallel: { ep: 64,  tp_attn: 1, dp_attn: 128, sp: 1 } }
```

约束（编译期 / 启动期断言）：

- $ep \leq routed_experts$ （每 rank 至少 1 个专家）
- $tp_attn \times dp_attn = N_npu$ （attention 维度划分覆盖所有 NPU；当 $tp_attn=1$ 时 $dp_attn = N_npu$ ）
- $tp_attn \leq 8$ （限同 PC16 内 full-mesh，§4.4.3 部署边界条款）
- $chunk_tokens \% m_hca == 0$ （§3.5 整除约束，与并行无关但启动期校验）

4.6 与 §3 KV cache 设计的协同（V4 Hybrid Attention 下重新核对）

§3 KV cache 决策	与 §4 并行化的关系
§3.4 per-(layer, chunk) striping 到所有 SSU	DP-attention 下，每个 prefix 的 KV 仅加载到 1 个 NPU；不同 prefix 自然分布到不同 NPU 与不同 SSU；SSU 与 NPU 两侧负载同时均衡
§3.5 chunk_tokens=512	与 attention 的 batch 维度无关；与 EP all-to-all 的 micro-batch 也无关；chunk 仅决定写时序、索引粒度、CSA / HCA / indexer 的 LBA 对齐
§3.6 单 meta server	DP-attention 下每个 NPU 都会向 meta server 查 prefix；查询 QPS $\approx$ 入口请求 QPS；鲲鹏 CPU 服务器单实例 v1 可承载
§3.3 V4 KV 缩到 5–7 GiB / 1 M-prompt	给"DP-attention 默认不开 TP"提供了关键支撑：V4 单 NPU HBM 完全容得下，scan 时延已被序列压缩消化掉了

核心一致性： DP-attention + per-(layer, chunk) SSU striping 让 *同一份 prefix KV 在整个集群中只存在一份、只加载一次到一个 NPU*；TP-attention (TP=k) 会让每 prefix 的 KV 加载 $\times k$ 份。在 V4 Hybrid Attention 下，TP 的收益不再为零（per-rank scan 真正缩 $1/k$ ，§4.4.2），但 代价 仍然 $\times k$ —— 因此 TP 是 **profile-driven** 的可调参，默认 TP=1，prefill 重负载下可调至 $2/4$ 。

4.7 Vocabulary Encoder / Decoder（embedding 与 LM head）部署

本节专指模型自身的两个矩阵：input embedding（vocabulary encoder：u32 token id $\rightarrow$ bf16 hidden 向量）与 LM head（vocabulary decoder：bf16 hidden 向量 $\rightarrow$ bf16 vocab logits）。它们与 §5.8 的 tokenizer / detokenizer **完全是两件事** —— tokenizer 在 F 上做 plain text $\leftrightarrow$ token id；本节的 embedding / LM head 在 NPU 上做 token id $\leftrightarrow$ vector / logits。

4.7.1 顶层选择

选项	是否选	理由
部署在 NPU HBM ，作为 model layer 0 / layer L+1	是 (v1 默认)	是模型本身的层；与其他 transformer 层共图、零中间跳转
部署在 host CPU，把 hidden 经 UB 传 NPU	否	多走 1 跳；浪费带宽 + μ s；HBM 上其实容得下
沿 vocab_size 维度 分片 到所有 NPU	否 (v1)； v2 候选	每 token 需 all-gather logits (V4 Pro batch=32 $\rightarrow$ 8 MiB / step)，代价远超存储节省
沿 vocab_size 维度 复制 到每 NPU	是 (v1 默认)	DP-attention 下零通信；存储成本 1-2 GiB / NPU 完全可承受

4.7.2 HBM 成本核算

V4 系列 vocab $\approx$ **128 K**（与 V3 同 tokenizer，待官方确认；与 §3.3 同口径）：

模型	vocab	hidden	embedding 矩阵 (bf16)	LM head 矩阵	DeepSeek 默认 tie weights	单 NPU 实占
V4 Pro	128 K	7 168	128 K $\times$ 7 168 $\times$ 2 B = 1.83 GiB	同上	是（共享同一矩阵）	1.83 GiB
V4 Flash	128 K	4 096	128 K $\times$ 4 096 $\times$ 2 B = 1.05 GiB	同上	是	1.05 GiB

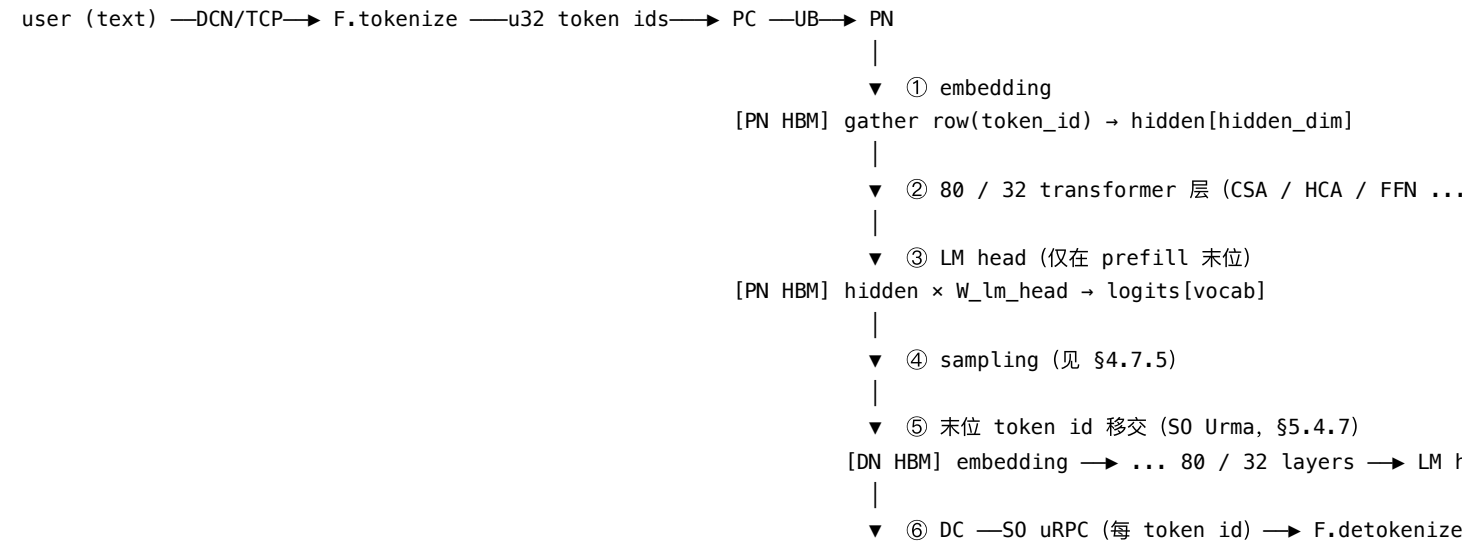
相对单 NPU HBM 总量与其他模型权重（数十 GiB 量级），1-2 GiB 占比 < 5 %，**v1 复制策略不构成压力**。

4.7.3 部署的 NPU 范围

阶段	用到的部分	部署的 NPU
prefill	embedding：把 prompt 的每 token 嵌入；LM head：仅在末位生成第一个 decode token 时用一次	PN 全部 NPU 各持一份
decode	embedding：每生成一个 token 嵌入一次；LM head：每生成 token 一次	DN 全部 NPU 各持一份

PN 与 DN 加载**同一个 model 文件**（含 embedding / LM head / 所有 transformer 层）；只是 prefill / decode 阶段对模型的使用模式不同。

4.7.4 数据流（与 §5.8 拼接成完整链路）

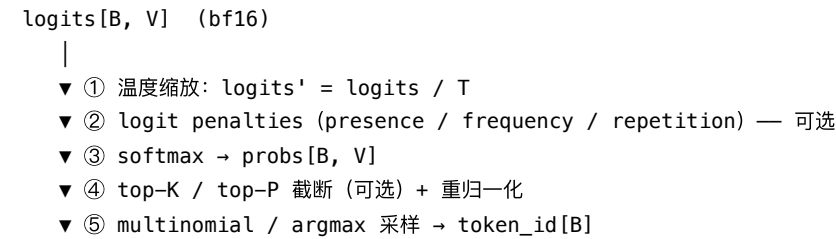


关键不变量：vocab encoder / decoder 完全在 NPU 内闭环；**logits 永远不出 NPU HBM**（仅最终采样后的 u32 token id 出 NPU），节省每 token ~8 MiB（V4 Pro batch=32）的跨 SO 流量。

4.7.5 Sampling（采样）：在 NPU 上完成，含温度的精确含义

LM head 输出 logits[batch, vocab]（bf16）后，立即在同一 NPU 上做 **sampling**，输出 u32 token_id[batch]。整个采样流水线写成一组 NPU kernel，无中间结果落 host。

5 步采样流水线（一次 forward 的尾巴）



① 温度缩放（temperature scaling）—— 详细说明

定义：温度 T 是采样前对 logits 做的标量除法，**直接控制采样分布的"尖锐度"**：

$$\text{logits}'_i = \frac{\text{logits}_i}{T} \Rightarrow p_i = \text{softmax}(\text{logits}')_i = \frac{e^{\text{logits}_i/T}}{\sum_j e^{\text{logits}_j/T}}$$

T 的语义：

T 取值	数学行为	模型表现
T → 0+	softmax 退化为 argmax；最高 logit 概率 → 1，其它 → 0	完全确定性贪心解码；同 prompt 总是同输出
T = 1.0	softmax 不变	"原始"模型分布；通常作为 baseline
0 < T < 1	分布更尖锐（高分 token 更容易被选）	更保守 / 更"精准"；适合代码、数学、事实问答
T = 1	原分布	自然多样性
T > 1	分布更平坦（次优 token 概率被抬高）	更发散、更"创意"；适合写作、头脑风暴

T 取值	数学行为	模型表现
$T \rightarrow \infty$	分布趋向 vocab 均匀	退化到完全随机采样

T = 0 的特殊处理：数值上不可除以 0；实现上 **T = 0 直接走 argmax 分支**（跳过 softmax + multinomial），**输出确定性 token_id = argmax(logits)**。这也是大多数 reproducible / deterministic 模式的实现路径。

T 与 top-K / top-P 的组合关系（步序很重要）：

原 logits $\rightarrow$ logits / T $\rightarrow$ top-K 截断（保留前 K 个 logit, 其余设 -inf）
 $\rightarrow$ top-P / nucleus 截断（保留累计概率 $\geq P$ 的最小集）
 $\rightarrow$ softmax $\rightarrow$ multinomial 采样

注意：**T 必须在 top-K / top-P 之前作用** —— 否则不同温度下 top-K 选出的相对概率比例不会变（破坏温度的语义）。

典型业务取值：

场景	推荐 T	说明
代码生成 / 数学 / 事实问答	0.0 ~ 0.3	强一致性；高 T 易引入"创造性"错误
通用对话	0.6 ~ 0.8	保留可读性的同时有自然多样
头脑风暴 / 创意写作 / 角色扮演	0.9 ~ 1.2	多样性优先
分类 / 评分类（取唯一 label）	T = 0	argmax；可重复

② Logit penalties（NPU 上向量化执行）

可选；按 batch 单独维护：

penalty 类型	公式	用途
repetition penalty (rep)	logits[i] /= rep if i ∈ history , else 不变	抑制重复（rep > 1 推开重复 token；< 1 反之）
presence penalty (pres)	logits[i] -= pres if i ∈ history	出现一次即扣分
frequency penalty (freq)	logits[i] -= freq * count(i, history)	按出现次数累计扣分
logit bias (用户自定义)	logits[i] += bias[i]	强制提升 / 屏蔽特定 token（如 banned words 设 -inf）

实现：每 batch 维护 history_set / history_count 在 NPU HBM 中（小数据结构），每步采样前一次 vectorized scatter-update。

③ Softmax（NPU 标准算子）

数值稳定版：probs = softmax(logits' - max(logits'))，避免 exp 溢出。bf16 输入 $\rightarrow$ fp32 softmax $\rightarrow$ bf16 probs（混合精度避免精度损失）。

④ Top-K / Top-P 截断

算子	描述
top-K	保留概率最高的 K 个 token，其余 prob 置 0；K 典型 1-100

算子	描述
top-P / nucleus	按 prob 降序累加，保留累计概率 >= P 的最小集；P 典型 0.7-0.95
min-P（v1.5 候选）	只保留 prob >= P × max_prob 的 token

截断后 重新归一化 probabilities，再走采样。

⑤ 采样（multinomial 或 argmax）

模式	触发条件	算法
argmax	T = 0 或 user 指定 do_sample=false	单 NPU op，纳秒级
multinomial	T > 0 默认	NPU 上的 cumsum + binary_search(rand) 实现；rand 由 NPU 端的 PRNG 生成（每请求 seed 可由 user 提供）

整批向量化

batch=32 时，5 步流水线作为一个 fused kernel 跑在 NPU 上：

量级	V4 Pro batch=32
logits 输入	32 × 128 K × 2 B = 8 MiB / step
中间 probs	32 × 128 K × 2 B = 8 MiB（HBM 内）
输出 token_id	32 × 4 B = 128 B
单 step 采样耗时	NPU 上 ~10-30 μs（含 fused kernel 启动）

整个采样过程在 NPU 上闭环；从未把 logits 送出 NPU。这是 §4.7.4 提到的"每 token 省 ~8 MiB 跨 SO 流量"的来源。

4.7.6 Sampling 参数的 per-request 配置

每条入口请求（OpenAI / DeepSeek API 兼容）携带自己的采样参数：

```
# F 收到的请求体 (HTTP/JSON) → 透传给 worker 的 sampling spec
sampling:
  temperature: 0.7      # T; T=0 → 自动走 argmax 分支
  top_k: 50             # 0 = 关闭
  top_p: 0.9           # 1.0 = 关闭
  repetition_penalty: 1.05 # 1.0 = 关闭
  presence_penalty: 0.0
  frequency_penalty: 0.0
  logit_bias: {}        # token_id → bias
  seed: null            # 给定则采样可重放
  do_sample: true       # false → 强制 argmax
  min_p: 0.0           # v1.5 候选
  stop: []             # stop sequences (在 F 上检测, §5.8)
```

per-request 配置如何下发到 NPU：F 收到请求后把 sampling spec 经 PC/DC（host CPU）放进 NPU mailbox；NPU 端的采样 kernel 每 step 读这份 spec 决定走哪个分支。spec 体积 < 1 KiB，相对 token 流量是噪声。

4.7.7 与 §4.6 的协同核对

§4 决策	与 §4.7 的关系
§4.1 DP-attention	让 vocab encoder/decoder 复制策略最优 （每 NPU 自给自足，无通信）
§4.3 EP-FFN 最大 EP	与 vocab 维度无关；FFN 走 EP，embedding/LM head 走 DP-replicate
§4.4 TP-attention 默认关闭	TP 也不影响 vocab；vocab encoder/decoder 不参与 attention head 切分
§3.3 V4 KV 1-2 GiB	embedding 1-2 GiB 与 KV 同量级；HBM 充裕，不互相挤占

4.7.8 v2 候选优化

优化	适用场景	收益 / 代价
沿 vocab 维度分片（vocab parallel）	vocab ≥ 1 M（如多语种 / 中英日韩文混合大词表）	节省 (1-2) GiB / NPU；代价：每 token 一次 all-gather logits
Speculative decoding 集成	高吞吐场景，提升每 step 接受 token 数	需要 draft model 的额外 LM head；与本设计正交
Constrained decoding（regex / JSON schema 强约束）	结构化输出	在 sampling kernel 前加 mask；与 logit_bias 同机制扩展

4.8 Continuous Batching 与会话生命周期

本节描述 V4 上的 **continuous batching (in-flight batching)**：新会话在 **token 边界**可以无缝加入正在执行的批，已结束的会话也在 token 边界释放槽位 —— 整个集群的 NPU 不会因为某个长序列还没出 EOS 而空转。本节聚焦四类事件：①温度参数施加 ②自回归（prefill vs decode）③新会话加入 batch ④已结束会话移除 batch；逐项给出 prefill 节点（PN/PC）与 decode 节点（DN/DC）的处理差异，并明确每步的 CPU 职责与 NPU 职责。

4.8.1 Continuous Batching 总览：4 步标准流程

步骤	动作	节点	网络 / 协议
① Request Arrival	新用户会话到达 service_access；F 完成 tokenize (§5.8) + PrefixMatch (§5.4.1)	user → F → M	DCN/TCP（外部）→ SO uRPC（内部）
② Prefill Phase	把不命中的 prompt 尾部经一次大 forward 算出全部 KV，写到 SSU；产生第一个 decode token	PN（在 host CPU PC 协调下）	SO Urma 写 SSU；SO uRPC 注册 ChunkRecord
③ Slot Insertion	调度器把会话从 prefill 切到 decode；DN 在自己的 slot table 中分配一个空闲 slot 给该会话；ChunkRecord refs 由 DC 注入 NPU mailbox	F → DC → DN	SO uRPC + UB
④ Token-Level Interleaving	DN 在下一个 step 同时为 batch 中 所有 active slots 各生成 1 个 token —— 包含老会话和刚加入的新会话	DN（每 NPU 自包含批）	SU Urma（EP/DP）+ SO Urma（KV 读）

V4 的**序列压缩** (§3.3.1 CSA + HCA) 让 slot 管理在 KV 字节维度上显著省内存：单 slot 的 KV 占 5-7 KiB / token（V4 Pro），而非 V3 MLA 的 ~70 KiB / token。同样 HBM 容量下可承载的并发会话数 **多 10x**。

4.8.2 温度参数（Temperature）的施加

术语澄清（先把"温度"两个语义分清）

"温度"含义	是什么	在本系统的位置	数据通路
采样温度（ mathematical sampling temperature, T ）【本节专指此】	per-request 的标量； §4.7.5 的 logits / T	每条请求自带；NPU 上 sampling kernel 消费	per-session sampling spec 经 mailbox 进 NPU
物理硬件温度（NPU die / SSU 散热温度）	硬件传感器读数	与模型输出 无关 （系统正常时）； 只供运维监控	DCN gRPC → Prometheus（§5.2.2 运维面）

关键不变量：模型输出分布**纯由数学采样温度决定，不受 NPU 物理温度等任何硬件传感器读数影响**——除非系统已发生硬件故障（位翻转、HBM ECC 不可纠错等）。这两套"温度"在 v1 系统中**走完全不同的网络与协议**，物理隔离，互不影响。

Prefill 节点的处理

- **场景：**prefill 末位生成第一个 decode token ("first token")
- **频率：**每会话 1 次
- **如何拿 T：**F 在 PrefixMatch 之后、Dispatch 之前已经从请求体中解析出 sampling spec（含 T）；经 SO uRPC 随 DispatchPrefill 一起送达 PC，PC 把 T 写入 NPU mailbox 中该会话对应的 sampling slot
- **NPU 上做什么：**prefill 末位 forward 完成 → §4.7.5 的 5 步采样流水线（含温度缩放）→ 第一个 decode token
- **复杂度：**单会话单 token 一次采样，相对 prefill 整体 forward 的几秒计算几乎是噪声

Decode 节点的处理（关键：**per-session** 矢量化）

- **场景：**每个 step 为 batch 内**所有 active slot** 同时采样 1 个 token
- **频率：**每会话每 step 1 次；batch 内每 slot 一组采样参数
- **如何拿 T：**每 slot 一份 sampling spec 常驻 NPU mailbox / HBM；DC 在会话注入时一次性下发，整个 decode 期间不变（除非用户中途调整）
- **NPU 上做什么：**sampling kernel 以 **batch_size × vocab** 的 logits 为输入，按 **per-row 温度 T[b]** 做向量化的 logits[b] / T[b]：

```
for b in 0..batch_size:           // NPU 上向量化执行
    if T[b] == 0:
        token_id[b] = argmax(logits[b])           // 短路 argmax 分支
    else:
        scaled = logits[b] / T[b]                 // 温度缩放
        # logit penalties / top_k / top_p / softmax / multinomial (详见 §4.7.5)
        ...
        token_id[b] = sample(scaled, ...)
```
- **batch 内不同会话可有不同 T**（甚至同一 batch 中既有 T=0 的代码请求又有 T=1.2 的创意请求），sampling kernel **不依赖全 batch 同 T**

Prefill 与 Decode 的核心差异

维度	Prefill 节点	Decode 节点
采样频率	每会话 1 次	每会话每 step 1 次
Batch 维度	单会话或小 micro-batch	大批（典型 32-128 slot）
Per-row 不同 T	可能不需要（多数 prefill batch 较小）	必须 per-row （slot 间彻底独立）
在端到端延迟中的占比	< 1 %（被 prefill forward 主导）	每 step 都触发，需高度优化 （fused kernel ~10-30 μs / step）

维度	Prefill 节点	Decode 节点
sampling spec 寿命	一次性	整会话期常驻（可能持续数千 step）

CPU vs NPU 分工

角色	做什么
F (CPU)	解析请求 JSON 中的 temperature 等 sampling 字段；validate ($T \in [0, 5]$ ，超界拒绝)；下发到 PC/DC
PC / DC (CPU)	把每会话的 sampling spec（含 T、top_k、top_p、penalty 数组）写到对应 NPU 的 mailbox slot；维护 slot ↔ session 映射表
PN / DN (NPU)	执行 §4.7.5 的 5 步采样流水线；per-row 读 T[b] 做 vectorized 缩放； 对外只输出 token_id[batch]，不输出 logits

4.8.3 Auto-regression：prefill 与 decode 的本质差异

Prefill 节点：序列内并行（不是 auto-regressive）

- 输入：unmatched_tail token 序列（一次性 N 个 token）
- 执行：N 个 token **一次性进 forward**（在序列维度并行），通过**因果 mask** 保证每个位置只看左侧
- 每层产出：所有 N 个位置的 KV → 一次性写到 SSU（按 chunk_tokens=512 切片，§3.4）
- 是 **auto-regressive** 吗：从模型语义上是（每位置只看左侧），但**计算上是并行的**（不需要逐 token 启动 kernel）
- 瓶颈：**算力 bound**（attention 与 FFN 都要算 N 次）；KV 写 SSU 顺序输出，不在关键路径

Decode 节点：严格串行的 token 循环

每 step 只前进 1 个 token（per slot）：

```
loop until session ends:
  1. NPU embed: token_id_n → hidden[hidden_dim]      (§4.7: embedding 表 gather)
  2. NPU forward 80 / 32 layers:
    - attention: 用本 slot 的所有历史 KV (含 prefix + 已生成 + 当前 step) 做 attn
    - FFN: EP all-to-all (SU) + MoE 计算
    - 累积 KV 到当前层
  3. 每 chunk_tokens=512 个 token 攒满一个新 chunk → DC 上报 meta server (§3.6.4 写入流程)
  4. NPU LM head + sampling: hidden → token_id_{n+1}  (§4.7.5)
  5. NPU 把 token_id_{n+1} 写回该 slot 的 mailbox
```

Prefill 与 Decode 的核心差异（Auto-regression 维度）

维度	Prefill	Decode
序列推进方式	一次性 N 个 token 并行	逐 token 串行
Kernel 启动次数	$O(\text{num_layers})$ （一次大 forward）	$O(\text{num_layers} \times \text{num_steps})$
计算 / 带宽 比	compute bound （FFN + attention 都满批）	memory bandwidth bound （每 step batch 小，FFN 几乎纯 HBM 扫描；§4.3.1）
KV 操作	写 （chunked，到 SSU）	读 + 增量写 （读历史 KV、写当前 step 的新 KV）
单 step 时延	不适用（粒度是整 prompt）	几十 ms / step（V4 Pro，128 NPU C 档）
主要耗时来源	attention 矩阵 N^2 + FFN N	FFN HBM 扫描（§4.3）

CPU vs NPU 分工

角色	Prefill 阶段	Decode 阶段
F (CPU)	路由 + 调度 (§5.5 T2-T5)	收 streaming 出来的 token id 做 detokenize (§5.8) + stop 检测
PC / DC (CPU)	PC: 维护 prefill 任务队列; 与 meta server 交互 (AllocateChunk / CommitChunk)	DC: 维护 slot table (slot ↔ session); 累积 chunk_tokens 满后向 meta server 提交; 从 NPU mailbox 取 token id 转发给 F
PN / DN (NPU)	PN: N 个 token 一次性 forward; 写 KV 到 SSU; prefill 末位采样 1 个 token	DN: 每 step 1 个 token 的 forward; attention 读 SSU + 用 KV cache; LM head + 采样

4.8.4 新会话加入 batch (Slot Insertion)

新会话经历的完整路径

T0 user 进站 → F.tokenize (§5.8) + PrefixMatch (§5.4.1) → 获得 ChunkRecord list + unmatched_tail
T5 F → PC: DispatchPrefill [SO uRPC]
T6–T13 PN 上 prefill: 读已命中的 KV (如果有) + 算尾部 + 写新 chunks
T14 PN → DN: 末位 token state 移交 [SO Urma]
T15 PC → F: prefill done ack
T16 F → DC: DispatchDecode (含 ChunkRecord refs + sampling spec) [SO uRPC]
T17 DC: 在自己 slot table 找一个空 slot; 写入会话元数据 + sampling spec → NPU mailbox
T18 DN: 下一 step 的 forward 把这个新 slot 也并入批, 与所有老会话一起算 1 个 token

Slot Table 数据结构 (DC 上)

```
SlotTable {
    capacity:    u32                // 典型 32 / 64 / 128
    slots: [Slot; capacity]
}

Slot {
    state:      enum {Vacant, Active, Pending_Prefill, Ending}
    session_id: u64
    model_id:   u32
    sampling_spec: SamplingSpec    // T, top_k, top_p, ...
    kv_chunk_refs: Vec<ChunkRef>   // ChunkRecord 引用
    last_token_id: u32             // 上一 step 输出
    tokens_emitted: u32            // 已生成 token 数
    max_tokens:  u32
    created_at_step: u64           // 加入 batch 时的 global step
}
```

NPU 端 Batch Mask 与 Token-Level Interleaving

每 step 的 forward 输入是 [batch_size, 1] 的 token_id 张量 + 一个 [batch_size] 的 active mask; NPU 的 attention/FFN/sampling 都按 mask 跳过 vacant slot:

```
input_token[b] = slot[b].state == Active ? slot[b].last_token_id : PAD
active_mask[b]  = slot[b].state == Active ? 1.0 : 0.0
```

老会话的"step 编号"和新会话的"step 编号"是相互独立的 (attention 只读各自 slot 的 KV history), 所以"老会话已经跑了 1 000 步、新会话才第 1 步"在同一 step forward 中完全和谐共存。

Prefill 与 Decode 的差异（加入会话维度）

维度	Prefill 节点	Decode 节点
"加入"事件	新会话整个 prompt 一次性进入 forward；不存在 mid-batch 加入	每 step 都可能有新会话进入（token 边界）
Batch 形态	单 prompt 或定向小批；同步触发	大 batch 持续运行；slot 异步进出
准备工作	读 ChunkRecord 拿到 prefix KV 引用；分配新 chunk 的 LBA	占据空 slot；从 SSU lazy load prefix KV（首次 forward 时按需读）
准备时间预算	与 prefill 自身耗时相比可忽略	必须赶在下一 step forward 前完成 mailbox 写入；典型 < 1 ms

CPU vs NPU 分工

角色	加入新会话的职责
F (CPU)	决定一个新请求归到 prefill 集群还是直接 decode（短 prompt 全 prefix hit 时可跳过 prefill）
PC (CPU)	prefill 排队、与 meta server 协作；prefill done 后通知 F
DC (CPU)	slot 分配（找 vacant slot）；写 sampling spec、ChunkRecord refs、初始 last_token_id 到 NPU mailbox；维护 session→slot 映射
PN (NPU)	完整 prefill forward；写 KV；末位采样
DN (NPU)	下一 step 自动把新 slot 纳入 batch；首次 forward 时按 ChunkRecord refs 经 SO Urma read 加载 prefix KV

4.8.5 已结束会话的移除（Slot Release）

会话结束的 4 类信号

信号	检测者	检测时机
EOS token	DN 采样输出 = EOS_TOKEN_ID；DC 读 mailbox 时识别	NPU sampling 后立即；DC 在下一 step 启动前响应
max_tokens 达到	DC 计数 tokens_emitted >= max_tokens	每 step 后
Stop sequence（文本级）	F 在 detokenize 后做 substring 匹配（§5.8）	F 检测到后经 SO uRPC 通知 DC
User cancel / timeout	F 检测连接断开 / 超时	异步

移除流程

```
T_end  DC 检测到结束信号
T_end+ε  DC 把 slot.state 设为 Ending; NPU 下一 step 不再为该 slot 算 forward
T_end+ε  DC → M: ReleaseRefs(req_id) [SO uRPC]
        (ChunkRecord refcount -= 1; 该 prefix 是否被淘汰由 §3.6.10 LRU 决定)
T_end+ε  DC → F: session_done [SO uRPC]
T_end+ε  F → user: 关闭 stream / 写 final response [DCN/TCP]
T_end+ε+δ  DC 把 slot.state 切到 Vacant; 可被新会话占据
```

关键设计点：KV 不立即擦除

会话结束时仅释放 ChunkRecord 引用计数；对应的 KV 字节仍留在 SSU 上（直到 §3.6.10 LRU 触发回收）。这是 prefix cache 的核心收益：刚结束的会话留下的 prefix 完全可能被下一个类似 **prompt** 的新请求**直接命中**（system prompt / 模板 / 长上下文场景常见）。

Prefill 与 Decode 的差异（移除会话维度）

维度	Prefill 节点	Decode 节点
"移除"事件	prefill 完成即"会话从 prefill 视角结束" → 移交 decode；不存在中途结束	会话可能在任意 step 结束；slot 异步释放
是否会引发 cache eviction	否（prefix 写入永远是"添加"）	否（释放只 -refcount，不直接 free）
GPU/NPU 利用率影响	prefill batch 小、独立	关键： 每 slot 提前结束都让 batch 有"洞"；下一 step 立刻用新会话填上 → 高利用率

CPU vs NPU 分工

角色	移除会话的职责
F (CPU)	文本级 stop sequence 检测 (§5.8)；user cancel / timeout 处理；通知 DC
DC (CPU)	数值级停止条件检测 (EOS、max_tokens)；状态机切换 slot.state；ReleaseRefs RPC；slot vacancy 簿记
DN (NPU)	mask 掉 Ending slot；下一 step 不再为它做计算；不主动 free 任何 KV（KV 由 meta server 管）
M (CPU)	refcount 减 1；触发 §3.6.10 eviction 检查（仅当容量超水位）

4.8.6 跨四类事件的 CPU / NPU 分工汇总

事件	F (CPU)	PC / DC (CPU)	PN / DN (NPU)	M (CPU)
温度施加	解析 / 校验 sampling spec	写 mailbox（per-slot T 等）	sampling kernel 中 per-row logits / T[b]	不参与
Auto-regression (prefill)	路由	AllocateChunk / CommitChunk	N 个 token 一次性 forward + 写 KV	LBA 分配 + ChunkRecord 注册
Auto-regression (decode)	detokenize / stop 检测	累积 chunk + 上报；token id 转发 F	每 step 1 token forward；读 SSU + sample	chunk 边界时 Allocate / Commit
加入新会话	tokenize + PrefixMatch	DC：分配 slot + 写 mailbox	下一 step 自动并入 batch	PrefixMatch + 引用计数 +1
移除已结束会话	文本 stop / cancel	DC：状态机 + ReleaseRefs	mask 掉该 slot；其余正常推进	refcount -1；按需 evict

4.8.7 V4 关键架构对 Continuous Batching 的支撑

V4 特性	对 continuous batching 的影响
CSA + HCA 序列压缩 (§3.3.1)	单 slot KV 占用从 V3 的 ~70 KiB / token 缩到 ~5-7 KiB / token；同 HBM 容量并发 slot 多 10x
DP-attention (§4.1)	每 slot 在单 NPU 内闭环算 attention，slot 间无通信；加入 / 移除任意 slot 都不影响其他 slot 的 attention 路径

V4 特性	对 continuous batching 的影响
EP-FFN (§4.3)	FFN 是 token 级路由 (top-K experts), slot 数量变化只改变 batch 形状, 不影响 expert routing 算法
per-(layer, chunk) SSU striping (§3.4)	新会话 lazy-load prefix KV 时, 从 ~num_layers 个不同 SSU 并行读, 吃满聚合带宽; slot 加入不引发热点
chunk_tokens=512 (§3.5.2)	chunk 边界粗 → decode 每 ~512 step 才有一次 chunk commit; slot 操作的 metadata RPC 频率低

4.8.8 Impact 总结（与用户给的 3 条 Impact 对齐到本系统）

项	在 V4 + 本系统下的具体含义
Temperature (数学采样温度)	纯数学操作；模型输出不受任何物理传感器（NPU 温度 / 风扇 / SSU 健康度）影响（系统正常时）。物理温度走 §1.7 / §5.2 的运维监控管线（DCN gRPC + Prometheus），与 model 输出物理隔离。 仅当硬件故障（HBM ECC 不可纠错、SO 链路中断等）才会反向影响输出 —— 这是错误，不是设计
Data Transfer (数据传输)	temperature 本身的传输几乎为 0（per-session 4 字节 spec，一次性下发常驻 mailbox）；continuous batching 的主要 KV / 权重带宽由 §3.4 SSU striping + §4.3 EP HBM 扫描承担 —— V4 的 CSA / HCA 已经把这两者的字节量做到 1/10（vs MLA），continuous batching 因此能容纳 10× 并发 slot
Batching（批化）	动态 + 连续：每 step 起点都允许新 slot 加入、已结束 slot 释放；NPU 永不因等待长尾会话而空转；GPU 利用率 typical > 90 %（V3-MLA 时代典型 60-80 %）

4.8.9 Config 钩子

```
batching:
  type:                continuous_inflight    # v1 默认; 不支持 static-batch / chunk-batch
  decode:
    max_active_slots:  64                    # DN 单 NPU 同时并发的 slot 上限
    slot_table_replicas: per_npu              # DP-attention 下每 NPU 自己一份 slot table
    new_slot_admit_at:  token_boundary        # 仅在 step 起点接纳新会话; step 内不可中断
    eos_drain_grace_steps: 1                  # 检测到 EOS 后再走 1 step (capture 完整 final hidden)
  prefill:
    micro_batch_size:   8                     # PN 上 prefill 的 micro-batch
    queue_depth:        32                    # PC 上 prefill 排队深度
    sampling_per_slot:  true                  # 每 slot 一份 spec; batch 内可异构 T / top_k / top_p
```

4.9 Multi-Token Prediction（MTP）

MTP 是 DeepSeek V3 引入、V4 延续并加深的“模型自带 multi-token 预测头”机制：模型主干末尾挂 D 个轻量“MTP module”，每个模块在主输出之上再预测一个未来 token。配合下一步的 verify forward，在保持主模型分布不变的前提下每次 forward 拿出 1 ~ (D+1) 个已确认 token，让 decode 显著提速（HBM 扫描成本被多 token 摊销）。

4.9.1 概念定位：MTP vs Speculative Decoding

维度	MTP（DeepSeek 集成式，本节）	通用 Speculative Decoding
Drafter	主模型自带的 MTP head（与主干联合训练）	独立的小 draft model（额外训练）
Drafter 容量	单层 transformer block-级（极小）	完整一份小模型（GiB 级）
HBM 占用	增量 ~几百 MiB / NPU	增量 GiB / NPU

维度	MTP (DeepSeek 集成式, 本节)	通用 Speculative Decoding
分布契合度	训练时强约束 → draft 与主模型分布天然接近	draft 分布与主模型有 gap, 命中率较低
接受率	高 (典型 70-90%)	中 (典型 50-75%)
与本系统兼容性	v1 直接支持 (与主模型一起加载)	留作 v2 扩展

4.9.2 D 的含义与"MTP4"的精确定义

术语	定义	例
D (MTP depth)	主输出之外额外 MTP module 数	DeepSeek-V3: D=1
MTP-D 系统每次 draft 产出的候选 token 数	D + 1 (主 LM head 1 个 + D 个 MTP 模块各 1 个)	DeepSeek-V3 一次 draft 出 2 个候选
"MTP4" (用户用词, 本文档采用)	总候选 4 个 (即 D = 3)	本节示例统一以 MTP4 (D=3) 展开

本设计文档为 V4 估算 **D = 3 / MTP4** (占位值, 待官方确认); 运行时 D 由 config 切换: `mtp.depth` ∈ {0, 1, 2, 3}, 0 表示关闭 MTP。

4.9.3 Draft 阶段：一次 forward 出 D+1 个候选 token

是的 —— 单次 decode forward 同时经主模型与所有 MTP 模块, 输出 D+1 个候选。

计算流 (MTP4 / D=3 为例)

```
input: last_committed_token  t_n
forward 主干 80/32 层
|
▼ hidden state at position n
|
└─→ 主 LM head — sample → draft_1 = t_{n+1}      (主输出, 永远存在)
|
▼ feed hidden to MTP-1 (输入 = main hidden + embedding(t_{n+1}))
|   MTP-1 内部一层小 transformer block + LM head
└─→ sample → draft_2 = t_{n+2}
|
▼ MTP-2 (输入 = MTP-1 hidden + embedding(t_{n+2}))
└─→ sample → draft_3 = t_{n+3}
|
▼ MTP-3 (输入 = MTP-2 hidden + embedding(t_{n+3}))
└─→ sample → draft_4 = t_{n+4}

draft sequence = [t_{n+1}, t_{n+2}, t_{n+3}, t_{n+4}]    (4 tokens)
```

增量计算成本

项	量级 (V4 Pro, hidden=7168, 估)
主干 80 层 forward	~1 (baseline)
MTP-1 / 2 / 3 (每个 ~ 1 层 transformer block)	共 +3 / 80 ≈ +3.75 % 计算
HBM 扫描	主权重 1 次 + MTP 权重 1 次 (很小)

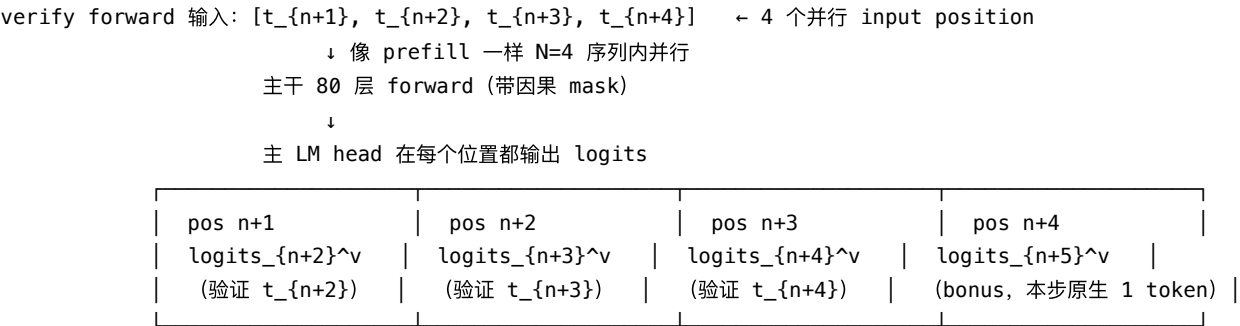
项	量级 (V4 Pro, hidden=7168, 估)
单 step 总耗时	与无 MTP 相比 +5-8 % (含 MTP 采样)

Draft 几乎不改变 forward 成本，但产出 token 数最多 **x4**，性价比极高（前提：acceptance 率高）。

4.9.4 Verify 阶段：下一次 forward 同时验证 D+1 个 token

是的 —— 把上一步产出的 4 个 draft token 作为 **4 个并行输入**，下一次 forward 一次性验证。

计算流 (V4 Pro 单 slot 视角)

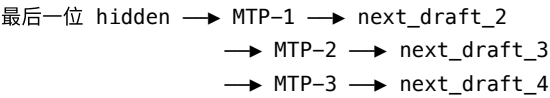


含义解读：

- 输入 position n+k 处的 logits 表示**主模型**基于 [t_{n+1} ... t_{n+k}] 在生成位置 n+k+1 时的分布
- 因此 logits^v[pos n+1] 用于验证 draft t_{n+2} 是否被主模型接受
- 最后一位 logits^v[pos n+4] 不验证任何 draft —— 它是**主模型**在 [t_{n+1}..t_{n+4}] 全被接受的假设下自然生成下一 token 的分布，称作 "**bonus token**"

同时附带新一轮 draft (pipelined)

verify forward 在主输出的最后一位也跑 MTP-1/2/3 模块：



这样 verify forward 同时完成两件事：① 验证上轮 4 个 draft；② 为下一轮再 draft 出 D=3 个新候选。**Pipeline 化、无空 forward。**

计算成本

维度	标准 decode (无 MTP)	MTP4 verify
单 forward 输入 token 数	1	4 (D+1)
主干 80 层算力	1x	4x (序列内并行；attention 是 4x4 因果)
FFN HBM 扫描	1x (每 token 都要扫一次权重)	1x (4 个 token 共享同一次扫描)
单 forward 耗时	T0	~T0 × 1.10-1.20 (attention 与 sampling 略增；FFN HBM 不变)
每 forward 产出已确认 token 数	1	1 ~ 4 (取决于 acceptance)

关键：V4 decode 是 **memory-bandwidth bound** (FFN HBM 扫描主导, §4.3.1)；输入序列长度从 1 增到 4 时，attention 计算变 $16\times (4\times 4)$ 因果矩阵) 但**绝对量级仍小**，FFN HBM 扫描不变。所以单 forward 耗时只略增 (~10-20%)，但**产出 token 数最多翻 4 倍**——这是 MTP 的核心收益。

4.9.5 Acceptance 规则：如何确立"接受"

每个 draft 位置 $t_{\{n+k\}}$ 是否被接受，用对应的 verify logits 判定。**两种模式**与 §4.7.5 sampling 的 T 设置直接对齐：

A. Greedy 模式 (T = 0 或 do_sample=false)

最简单：

```
for k = 1, 2, ..., D:                // 即 t_{n+2}, t_{n+3}, t_{n+4}
  if argmax(logits^v[pos n+k]) == draft_{k+1}:
    accept token k+1
  else:
    reject from k+1 onwards (含 k+1)
    break
```

接受逻辑是"最长前缀严格相等"。

B. Stochastic 模式 (T > 0, 标准 speculative-sampling)

在保持**主模型分布不变**的前提下做拒绝采样 (Leviathan et al. 2023 / Chen et al. 2023 的经典 spec sampling 推导)：

```
对每个 draft position k = 1, ..., D (验证 t_{n+k+1}):
  p = softmax(logits^v[pos n+k] / T)           // 主模型在该位置的真实分布
  q = MTP_draft_dist_at_position[k]           // draft 时 MTP 模块给出的分布 (同 T)
  accept_prob = min(1, p[draft_{k+1}] / q[draft_{k+1}])
  r ~ U(0,1)
  if r < accept_prob:
    accept draft_{k+1}
    continue
  else:
    # 拒绝；从主模型的"残差分布"重采样这一位
    residual = max(p - q, 0)
    residual /= sum(residual)                  // 归一化
    committed_at_pos_n+k+1 = sample(residual)
    STOP (后续 draft 全部丢弃)
```

```
如果全部 D 个 draft 都被接受：
  再加上 bonus token = sample(softmax(logits^v[pos n+D] / T))
```

数学保证：经过 spec-sampling 的拒绝采样规则后，**最终 commit 的 token 序列分布严格等于主模型逐 token 自回归采样的分布**。MTP 不引入任何采样偏差。

接受数量统计

输出场景	committed token 数
全部 D 个 draft 接受 + bonus	D + 1 (满分；MTP4 = 4)
接受前 j 个 draft (j < D)，第 j+1 个被拒 → 用 residual 重采样为新 token	j + 1
第 1 个 draft 就被拒 → 用 residual 重采样	1

即使最坏情况，**每 forward 也至少产出 1 个 token** (与无 MTP 相同)，从不会"白跑"。

4.9.6 KV cache 在 MTP 下的处理

暂存 → 提交 (speculative KV)

verify forward 处理 4 个 input position 时，主干每层都产出 4 个位置的 KV。这些 KV 暂存在 NPU HBM 的 scratch 区，不立即写到 SSU；待 acceptance 判定后：

```
suppose accepted prefix length = j  (0 <= j <= D)
committed positions = n+1 .. n+j+1    (含 bonus, 共 j+1 个 token)

for layer in all_layers:
    commit  scratch_KV[layer][positions n+1 .. n+j+1]    to chunk buffer
    discard scratch_KV[layer][positions n+j+2 .. n+D+1]
```

与 §3.4 chunk-based KV 写流程的兼容

- 仍按 chunk_tokens=512 边界 commit；committed 长度 (j+1) 每次 += 给 slot 的 chunk 累积量
- chunk 跨边界时仍走 §3.6.4 AllocateChunk + CommitChunk 流程
- 因为 committed 长度按 token 增长（不是固定 4），chunk 边界判断需要按"committed_count % chunk_tokens == 0"触发，不是"forward_count % (chunk_tokens / mtp_size) == 0"

拒绝时的 scratch 回收

scratch buffer 是 NPU HBM 内的预分配环形缓冲，固定 D+1 个 token slot 的容量；overwrite 上一步残留即可，无需主动 free。

4.9.7 对 decode 的影响（量化）

期望 token / forward（接受率链式衰减）

设单步 acceptance 概率 p_acc（典型 V3 报道 0.85）。则一次 verify 后接受 token 数（含 bonus）的期望：

E[committed | MTP4, p_acc=0.85]

= 1 (bonus 永远有)

+ sum_{j=1..3} P(前 j 个全接受) × 1

= 1 + 0.85 + 0.85² + 0.85³

≈ 1 + 0.85 + 0.72 + 0.61

≈ 3.18 tokens / forward

MTP 配置	p_acc=0.85 时期望 token/forward	单 forward 时间增量	有效 throughput 提升
D=0 (关 MTP)	1.00	0%	1×
D=1 ("MTP2")	1.85	+5%	~1.76×
D=2 ("MTP3")	2.57	+10%	~2.34×
D=3 ("MTP4")	3.18	+15%	~2.77×
D=4	3.61	+20%	~3.01×（边际收益递减）

p_acc = 0.85 是 V3 报告中 MTP-1 的实测；V4 加深 D 后链路上端 acceptance 略降，估 0.80-0.85。MTP4 在 V4 下预计 ~2.5-2.8× decode throughput。

HBM 带宽摊销（核心收益）

decode 是 memory-bandwidth bound (§4.3.1)。MTP 的优势是让一次 HBM 扫描出多个 token：

指标	无 MTP	MTP4 (avg 3 commit)
单 forward FFN 权重 HBM 扫描	1 次 / 1 token	1 次 / 3 token
单 token 摊到的 HBM 扫描	1 次	0.33 次
每 token 时延	T_step	T_step / 3 (粗算)

对 continuous batching 的影响

维度	行为
Batch 内 slot 独立性	完全独立：每 slot 自己一份 draft / verify / accept；可异构 D（虽然实现上常统一）
Slot 加入 batch	新 slot 第一次 decode 走"无 draft → 单 token bonus"（其实就是普通 decode），下一步开始才有 draft；有 1-step warm-up
Slot 离开 batch	当 commit 序列出现 EOS / max_tokens 时立即停；可能在 verify 中段命中 → 只 commit 到 EOS 那一位（截断），不浪费
Stop sequence 检测	F 在 detokenize commit 后做 (§5.8)；可能 commit 多 token 才检出 stop —— 已超出的 token 在 F 端 丢弃 （不返回给 user），同时 DC ReleaseRefs

端到端 TTFT 不受影响

MTP 仅作用在 decode；**prefill 完全不变**（prefill 末位仍只产出 1 个 first token）。TTFT 与 §3 设计一致。

4.9.8 与 §4.7 sampling、§3.4 KV、§5.5 时序的接口

与 §4.7.5 sampling 的对齐

verify pass 的接受规则**复用 §4.7.5 的同一份 sampling 参数**（T、top_k、top_p、penalties），保持采样行为一致：

§4.7.5 步骤	在 MTP 中的位置
温度缩放 logits / T	draft 时各 MTP 模块 + verify 时主 LM head 各位置 logits 都按 T[b] 缩放
logit penalties	同 §4.7.5；按 slot history 应用
top-K / top-P	同 §4.7.5
multinomial / argmax	T=0 → argmax-based greedy 接受；T>0 → spec-sampling rejection 接受

与 §3.4 KV chunk 的对齐

- chunk_tokens=512 不变
- 每 step commit 的 token 数从 1 升到 1-4（MTP4）；chunk 边界检查改为"committed % 512 == 0"

与 §5.5 时序图的差异

T18-T22 的 decode 循环改写为：

```

T18  DN: forward([previous bonus token + last_step's drafts]) → main logits + new MTP drafts
T19  DN: rejection-sampling acceptance → committed[1..k+1]
T20  DN: 把 committed 写 KV scratch; scratch 中拒绝部分丢弃
T21  DN: 累积 commit 总数; 满 chunk 触发 §3.6.4 写流程
T22  DN → DC: u32 token id × (k+1) → DC → F → user (流式 token)
```

每 forward 不再是"1 个 token / 步", 而是"**1 ~ D+1 个 token / 步**".

4.9.9 V4 关键参数预估

参数	预估值（占位）	来源
MTP depth D	3 （MTP4 总产出 4 token）	V4 估，待官方
每 MTP module 大小	~1 transformer block（hidden=7168 / 4096, ~ 100-300 MiB）	预估
单 NPU MTP 模块总占用（V4 Pro D=3）	~600-900 MiB	估算
单 NPU MTP 模块总占用（V4 Flash D=3）	~300-450 MiB	估算
draft 阶段计算增量	+3.75 ~ +5 % FLOPs	算术：D 层 / num_main_layers
verify 阶段单 forward 时延增量	+10 ~ +20 %	input_len 1→4 attention 开销
期望 acceptance（V4, D=3, T=0.7）	0.80-0.85	类比 V3 报道
期望 throughput 提升（C 档 128 NPU）	~2.5-2.8x	综合

4.9.10 Config 钩子

```
mtp:
  depth:          3          # D; 0 = 关 MTP; 1/2/3 对应 MTP2/3/4; 运行期可改
  acceptance_mode: auto     # auto = 按 sampling.T 选 greedy/stochastic
                           # greedy / stochastic 也可强制
  scratch_kv_slots: 4       # NPU HBM 中预留 D+1 = 4 个 token 位的 scratch
  per_request_override: true # 请求体可携带 mtp.depth 覆盖（如纯 deterministic 场景关 MTP）
  warmup_step_after_admit: 1 # 新 slot 加入后第 1 step 不做 draft（无前序 hidden）
  metrics:
    expose_acceptance_rate: true # 输出每 slot / 全 batch 的 acceptance histogram
```

4.9.11 与 §4.8 Continuous Batching 的协同小结

协同点	说明
同一 batch 内多 slot	每 slot 独立 draft / verify / commit；NPU sampling kernel 按 batch 维度向量化
Slot 加入	第 1 step 无 draft（warmup），第 2 step 起跑 MTP4 全速
Slot 退出	EOS 出现在 committed 的第 j 位 → 仅 commit 到第 j 位；其余 (D - j) 个 draft 视为浪费但 不影响其他 slot
Stop sequence	F 在 detokenize commit 后检；可能跨 step / 跨 token 触发；DC 收到 F 通知后立即 mark slot 为 Ending
KV 写 SSU	仅 commit 部分进 chunk buffer；按 §3.4 chunk_tokens=512 触发提交
MTP 关闭场景	mtp.depth=0 → degrade 到无 MTP 的 1 token / forward；其他系统机制不变

5. 节点交互矩阵与通信模型

本章给出系统中所有逻辑节点两两之间的交互关系：是否有直接通信、走哪类网络、传什么内容、量级如何。目的是把 §1（硬件）、§3（KV cache）、§4（并行）的所有数据流和控制流在一张表上闭合，并解释为什么"EP / DP 走 SU、其余走 SO"是必要的隔离规则。

5.1 节点清单

系统有 7 类逻辑节点（每类可有 1 ~ N 个实例，规模见 §1.6 三档场景）：

编号	简称	物理载体	角色
①	F	鲲鹏 CPU 服务器 (service_access)	前端接入 + 调度器（合一）：外部 HTTP/gRPC ingress (DCN)、tokenize、调用 meta (SO uRPC)、把任务派发给 PC/DC (SO uRPC)、把响应回流给 user
②	M	鲲鹏 CPU 服务器 (kv_meta)	KV Meta server：维护 radix tree + LBA 分配 (§3.6)
③	PC	PC16 host CPU (prefill cluster)	Prefill PC16 主机 CPU：代 NPU 发 RPC (AllocateChunk / CommitChunk)、与 NPU 走 doorbell / mailbox
④	PN	PC16 NPU (Ascend 950, prefill cluster)	Prefill NPU：算 prompt → KV， 直接 经 SO 写 SSU
⑤	DC	PC16 host CPU (decode cluster)	Decode PC16 主机 CPU：与 PC 同分工，但服务 decode
⑥	DN	PC16 NPU (Ascend 950, decode cluster)	Decode NPU：DP-attention + EP-FFN， 直接 经 SO 读/写 SSU
⑦	S	SSU12 内的 SSU 单元	KV 字节存储 (LBA-direct, §3.2)，被动接受 SO 上的读/写

v1 三档场景下：F = 1、M = 1、PC ∈ {1, 4, 8}、PN ∈ {16, 64, 128}、DC ∈ {1, 4, 8}、DN ∈ {16, 64, 128}、S ∈ {1, 1, 144}（见 §1.6）。

5.2 网络选择原则

5.2.1 协议栈速览：UB / Urma / uRPC vs RoCE / gRPC

系统中并行存在两套 RPC 栈，**按物理网络选用**（与 §1.3 末尾的硬约束一致）：

物理网络	链路 / 传输层	RPC 协议	典型场景	RTT 量级
SU (UBL128 域内 NPU↔NPU)	UB / UBG	(只走 Urma 数据面, 无 RPC)	EP / DP all-to-all	< 10 μs
SO (跨 UBL128 any-to-any)	UB / UBG	uRPC over Urma	内部 hot-path: PrefixMatch / AllocateChunk / CommitChunk / Dispatch; NPU↔SSU 数据面则用 Urma read/write (非 RPC)	< 100 μs (RPC)
DCN/RoCE (CPU 通用以太网)	Ethernet / TCP/IP	gRPC (HTTP/2 + protobuf) 或 JSON/HTTP	外部 user ingress; 运维 / 监控 / 部署 / 健康检查; POSIX FS (非 RPC)	~ms
服务器内 UB (NPU↔CPU NUMA)	UB	(doorbell + mailbox, 无 RPC)	启动 kernel / 完成中断	< μs

物理网络	链路 / 传输层	RPC 协议	典型场景	RTT 量级
内)				

协议选用规则（v1 硬约束）：

- 走 DCN 的 RPC 一律 gRPC 或 JSON/HTTP（与外部生态、Kubernetes、Prometheus 等无缝对接）
- 走 SO/SU（UB 网络）的 RPC 一律 uRPC over Urma，不使用 gRPC
- 上层服务先选网络，再决定协议：hot-path 选 SO（→ uRPC）；运维 / 监控 / 外部接入选 DCN（→ gRPC/HTTP）

为什么 hot-path 选 uRPC 而不是 gRPC：gRPC over RoCE Ethernet 要经 TCP/IP + HTTP/2 + protobuf 三层栈，单跳 RTT 在 ms 级；uRPC 直接基于 Urma 的可靠传输，省掉 TCP/HTTP，RTT 缩到 < 100 μs，与 prefill / decode 的 chunk 粒度匹配。但这并不"取代" gRPC — gRPC 仍然是 DCN 上的标准协议，运维 / 监控 / 外部接入这类不在数据 / 计算关键路径上的服务调用继续走 DCN gRPC。

5.2.2 网络选择规则（v1 强制）

参考 §1.3 的三类物理网络（SU = UBL128 域内 NPU↔NPU 3.2 Tbps；SO = 跨 UBL128 NPU/CPU/SSU any-to-any 400 Gbps；DCN = 外部接入 + 基础设施数据面）和服务器内部 UB（UB = NPU↔CPU 8×400 Gbps，PC16 服务器内 NUMA 内）。

流量类别	走的网络	协议层	原因
EP all-to-all (FFN 专家路由)	SU (同 UBL128) / SO (跨 UBL128 时不得已)	Urma	高频、大流量、对延迟极敏感 (每 token 每层 2 次 dispatch+combine)；优选独占的 SU 域内带宽
DP-attention 同步 (barrier / all-reduce)	SU (同 UBL128) / SO (跨 UBL128)	Urma	同上
KV 字节读 / 写 (NPU ↔ SSU)	SO	Urma read/write	SSU 不在 SU 域内；且不能与 EP/DP 抢 SU
跨集群 NPU 间数据交付 (如 prefill→decode 末位 activations 移交)	SO	Urma	不在 SU 域内
内部 hot-path RPC (F↔M PrefixMatch、F↔PC/DC Dispatch、PC/DC↔M Allocate/Commit)	SO	uRPC over UB Urma	μs 级 RTT，与计算粒度匹配；与数据面 Urma 共物理网但独立 QP 隔离
NPU ↔ host CPU 的 doorbell / mailbox	服务器内部 UB (NUMA 内)	UB	仅 PC16 内部，不出服务器
外部 user ingress	DCN (TCP/IP 段, §1.7)	gRPC 或 JSON/HTTP	外部 IP 世界标准接入
运维 / 管理 / 监控面 (部署、配置下发、健康检查、Prometheus 指标拉取、SSH、日志收集 ...)	DCN	gRPC 或 JSON/HTTP	非 hot-path、ms 级 RTT 可接受；与外部运维生态 (k8s / Prometheus / Grafana / Loki 等) 无缝对接
POSIX FS / 镜像	DCN	NFS / S3 等	§1.7 基础设施数据面，与系统数据 / 控制路径解耦

最高优先级原则（用户硬约束）：

EP / DP 流量必须独占 SU；任何其他类型的 NPU↔NPU、NPU↔SSU、NPU↔CPU 数据流量、以及 CPU↔CPU 的 uRPC 控制面都不得占用 SU，以防止 SU 上 EP/DP 的延迟出现抖动 (jitter)。

即便在某场景中 prefill 与 decode 集群恰好处于同一 UBL128 内、SU 直连可达，**仍然把它们的数据交付走 SO**，绝不让"非 EP/DP"流量沾染 SU。

SO 上的协议分层（控制面与数据面都在同一物理网络，但 Urma QP 天然隔离）：

SO 上的流量	占用
数据面 Urma（KV bytes、跨域 EP/DP、PN→DN 移交）	大块、高带宽（GiB/s 量级）
控制面 uRPC	小包、低带宽（< 100 MiB/s 量级，§5.6 论证）
隔离机制	不同 Urma queue pair；uRPC 用独立的 RC QP；调度互不影响

5.3 7×7 节点交互矩阵：按网络类型

行 = 发起方，列 = 被调方；单元格 = 走的物理网络（协议层），— 表示无直接交互。矩阵对称的部分仍逐对填写，因为发起方向不同时含义不同。

发起 \ 被调	F	M	PC	PN	DC	DN	S
F	SO uRPC (HA, v1 单实例)	SO uRPC (PrefixMatch / ReleaseRefs)	SO uRPC (dispatch prefill task)	—	SO uRPC (dispatch decode task)	—	—
M	SO uRPC (异步 capacity hint)	—	SO uRPC (AllocateResp / Heartbeat)	—	SO uRPC (AllocateResp / Heartbeat)	—	—
PC	SO uRPC (task ack / metric)	SO uRPC (AllocateChunk / CommitChunk / ReleaseRefs)	—	UB (doorbell / mailbox)	—	—	—
PN	—	—	UB (completion 中断)	SU Urma (同 UBL128 的 EP/DP)； SO Urma (跨 UBL128)	—	SO Urma (prefill→decode activations 交付)	SO Urma write (KV 字节写)
DC	SO uRPC (task ack / metric)	SO uRPC (AllocateChunk / CommitChunk / ReleaseRefs)	—	—	—	UB (doorbell / mailbox)	—

发起 \ 被调	F	M	PC	PN	DC	DN	S
DN	—	—	—	SO Urma (prefill→decode 反向 ack, 可选)	UB (completion 中断)	SU Urma (同 UBL128 的 EP/DP); SO Urma (跨 UBL128)	SO Urma read (KV 字节读)
S	—	—	—	SO Urma (KV 字节响应给 PN, 少见, prefix hit on prefill 时)	—	SO Urma (KV 字节响应给 DN, 主流量)	—

额外：外部 user → F 的入口流量走 **DCN (HTTP/gRPC over TCP/IP)**，不在 7×7 矩阵内。

矩阵的几个不变量：

- 1. **NPU 永远不直接说 uRPC**：F、M 列里 PN / DN 行全是 —；PN/DN 不承担 uRPC stack，所有控制面调用经其本地的 PC / DC 中转。
- 2. **CPU 永远不在 KV 数据路径上**：PC / DC 列里 S 行全是 —；F、M 列里 S 行也是 —。SSU 字节流量只与 NPU 直连。
- 3. **SU 仅承载 NPU↔NPU 的 EP/DP Urma 流量**：矩阵中 SU 单元格只出现在 (PN, PN) 与 (DN, DN) 对角，且明确加了"同 UBL128"限定；其余 NPU↔NPU 仍走 SO。
- 4. **SO 同时承载控制面 uRPC + 数据面 Urma**：但二者通过独立 Urma QP 隔离调度（控制面短包不会被大块 KV read/write 阻塞）。
- 5. **DCN 已退出系统内部**：矩阵中找不到任何 DCN 单元格；DCN 只承担外部 ingress 与基础设施 (§1.7)。

5.4 各对节点的交互细节

下面只列有**直接交互**的节点对（按上表标 ≠ — 的方向）。对每一对给出：发起方、内容、协议 / 格式、频率与量级、同步语义。

5.4.1 F ↔ M：前端 ↔ Meta server (SO uRPC)

项	取值
调用方向	F → M (PrefixMatch)；M → F (resp + 异步 capacity hint)
协议	uRPC over UB Urma ，承载在 SO 网络
内容	PrefixMatch(token_seq) → ChunkRecord[] + unmatched_tail_len
频率	每入口请求 1 次
单次量级	请求 ~几 KiB (token id list 或 hash 序列)；响应 1 600 B / chunk × 2 048 chunks ≈ 3 MiB （最坏 V4 Pro 1 M context 全 hit)
同步语义	同步 (F 阻塞等响应才能调度 worker)
关键路径	在端到端请求 TTFT 上；uRPC μs 级 RTT 让此调用对 TTFT 几乎无贡献

5.4.2 F ↔ PC / F ↔ DC：调度派发 (SO uRPC)

项	取值
调用方向	F → PC (DispatchPrefill)；F → DC (DispatchDecode)；PC/DC → F (task done / metric upstream)

项	取值
内容	DispatchPrefill(req_id, prompt_tokens, ChunkRecord[], target_npu_set, model_cfg)
频率	F → PC: 每请求 1 次; F → DC: 每请求 1 次
单次量级	request 几十 KiB (含 ChunkRecord list + token tail); response 同量级
同步语义	F → PC 异步触发; PC 完成后异步 ack (解耦)

5.4.3 PC ↔ M / DC ↔ M：写元数据（SO uRPC）

项	取值
调用方向	PC/DC → M (AllocateChunk、CommitChunk、AllocateWindow、CommitWindow、ReleaseRefs、Heartbeat)
协议	uRPC over UB Urma (详见 §3.6.4 接口定义)
频率	每个新 chunk 一对 (Alloc + Commit); 512 token / chunk → 每 ~512 token 触发 2 RPC
单次量级	AllocateReq ~1 KiB; AllocateResp ~1.6 KiB (Pro, 80 LayerBlock); CommitReq ~1.6 KiB
同步语义	Allocate 同步阻塞 (NPU 等到 LBA 才能写 SSU); Commit 可异步 (NPU 已写完, 对查询可见性可延迟数 ms)
路径影响	Allocate 在 prefill / decode 关键路径上; 目标 < 100 μs RTT (uRPC over UB Urma, 单跳 SO 交换)

5.4.4 PC ↔ PN / DC ↔ DN：服务器内 host ↔ NPU（UB）

项	取值
协议	NUMA 内 NPU↔CPU 8×400 Gbps UB (§1.2)
内容	doorbell (host → NPU 启动 kernel)、mailbox (NPU → host 完成中断)、ChunkRecord list 注入、tokenized prompt 注入
频率	每 chunk 一对 ("开始写"+"写完"); 每请求一次"task 起始"
单次量级	控制信号 < 1 KiB; token / metadata payload 几 KiB ~ 几十 KiB
同步语义	doorbell 异步; 中断异步
不出服务器	完全在 PC16 内部, 不上 SO/SU/DCN

5.4.5 PN ↔ PN / DN ↔ DN：NPU 之间的 EP / DP 流量（SU 优先，跨域才走 SO）

项	取值
协议	UB Urma (SU 单层无阻塞交换) / UB Urma over SO (跨 UBL128)
内容	EP all-to-all (dispatch + combine); DP-attention 同步 (barrier / 必要时 all-reduce)
频率	每层 2 次 all-to-all × num_layers × tokens_per_step ; DP barrier 每层 1 次
单次量级 (V4 Pro 单 token)	dispatch ~150 KiB; combine 同量级 (§4.3.4)
总流量	C 档 (128 NPU) decode 单 step: 每 NPU 每秒 ~10 GiB 级 EP 流量
同步语义	强同步 (barrier 形式)
网络路由	同 UBL128 → SU (首选) ; 跨 UBL128 → SO (不得不用, 并提示告警)

项	取值
SU 独占性	本类流量是 SU 网络的唯一合法占用方 (§5.2 硬约束)

5.4.6 PN → S / DN ↔ S: NPU ↔ SSU 的 KV 字节流量 (SO 唯一)

项	取值
协议	UB Urma read/write 直接读/写 SSU 的 LBA 段 (无文件系统, §3.2)
内容	urma_read(ssu_id, lba_start, lba_count) → KV bytes; urma_write(ssu_id, lba_start, lba_count, payload)
发起方	NPU 本身 (DMA / SO 引擎), 不经 host CPU
频率	prefill: 每 chunk 80 个 SSU 写 (V4 Pro, 每层 1 次); decode: 每 attention scan 80 个 SSU 读 (§4.6 一份 KV 一次加载)
单次量级	CSA 层 chunk 148 KiB (Level 1 拼接后 1 IO, §3.4.8); HCA 层 chunk 4.5 KiB ; 滑动窗口段每 prefix 一次 144 KiB / 层
单 prefix 1 M-context 全召回	V4 Pro ~7 GiB / V4 Flash ~2.4 GiB (详见 §3.4.6)
同步语义	异步 SO Urma; NPU 用 Urma completion queue 等齐
网络路由	SO 唯一 (SSU 不在 SU 域内)
隔离性	与 SU 上的 EP/DP 物理隔离 (不同网卡 / 不同交换平面), 互不干扰; 与同 SO 上的 uRPC 控制面通过独立 QP 隔离

5.4.7 PN ↔ DN: prefill → decode 数据移交 (SO, 少量)

项	取值
协议	UB Urma over SO (小块同步移交)
内容	prefill 计算完最后一段后, 把"末位 token 的 KV / 末位激活 / next-token logits"移交给 decode worker; decode 用此作为自回归起点
频率	每请求 1 次 (prefill→decode 切换时)
单次量级	几 KiB ~ 几 MiB (仅末位 1 个 token 的少量 state)
同步语义	同步 (decode 必须等 prefill 末位)
网络路由	SO , 即便 prefill 与 decode 同 UBL128 也不走 SU (§5.2 隔离性硬约束)

额外说明: 在 §3 的 KV 经 SSU 流转的设计下, prefill→decode 的 KV 主体已经通过 SSU 中转 (prefill 写入、decode 读出), 所以 PN→DN 的直接 SO 数据交付实际上**只承载末位 token 的少量 state**, 是一个轻量同步点。

5.4.8 隐式 / 边界情况

节点对	状态	说明
F ↔ F	SO uRPC (v1 单实例时无)	HA 多实例时 leader 选举、负载均衡走 SO uRPC
M ↔ M	—	v1 单实例; v1.5 分片时按 prefix_id hash 走 SO uRPC

节点对	状态	说明
PC ↔ PC	—	同集群多 prefill 主机不直接通信；都通过 F 协调
DC ↔ DC	—	同上
PC ↔ DC	—	prefill→decode 移交由 F 编排，不走直连 CPU 路径
S ↔ S	—	v1 replication_factor=1，无副本同步；v1.5 多副本时走 SO Urma
F ↔ PN / F ↔ DN	—	F 是 CPU；uRPC 控制面只到 PC/DC，不直击 NPU
M ↔ PN / M ↔ DN	—	同上：NPU 不承担 uRPC stack
F / M ↔ S	—	元数据节点不在数据路径上；M 对 SSU 的"知识"由其内部 free list 表示，无需直接访问 SSU

5.5 端到端请求生命周期（典型路径，附时序）

下面给出一条 prompt 进入系统直到首 token 返回的完整路径，每步标注网络与节点。

```
T0 user → F                                [DCN/TCP, HTTP/gRPC]   HTTP request
T1 F: tokenize
T2 F → M: PrefixMatch(token_seq)           [SO uRPC]
T3 M → F: ChunkRecord[] + unmatched_tail    [SO uRPC]
T4 F: choose prefill / decode worker set
T5 F → PC: DispatchPrefill(unmatched_tail, ChunkRecord[], target_PN_set)  [SO uRPC]
T6 PC → PN: doorbell + ChunkRecord list     [UB (PC16 内部)]
T7 PN → S: Urma read 命中部分 KV bytes (按 ChunkRecord)  [SO Urma read]
T8 PN: 算 unmatched_tail 各 chunk 的新 KV
T9 PN ↔ PN: EP all-to-all (每层 2 次) + DP-attention barrier
                                           [SU Urma (同 UBL128) / SO Urma (跨 UBL128)]
T10 PC → M: AllocateChunk(...)              [SO uRPC] ← 每 chunk 完成时
T11 M → PC: LayerBlock[]                   [SO uRPC]
T12 PN → S: Urma write 新 chunk KV bytes    [SO Urma write]
T13 PC → M: CommitChunk(...)               [SO uRPC]
T14 PN → DN: 末位 token state 移交         [SO Urma] ← prefill 收尾
T15 PC → F: prefill done ack               [SO uRPC]
T16 F → DC: DispatchDecode(...)            [SO uRPC]
T17 DC → DN: doorbell                     [UB]
T18 DN → S: Urma read KV (每层 1 次)       [SO Urma read]
T19 DN ↔ DN: EP all-to-all + DP-attention [SU Urma / SO Urma]
T20 DN: 生成下一 token logits → 采样 → 累积 chunk
T21 (累积满 chunk) DC → M: AllocateChunk → LayerBlock → DN → S: Urma write → CommitChunk
                                           [SO uRPC + SO Urma]
T22 DN → DC → F → user: token stream       [UB → SO uRPC (DC→F) → DCN/TCP (F→user)]
T23 (重复 T18~T22) 直到 E0S
T24 F → M: ReleaseRefs(req_id)             [SO uRPC] ← 释放 ChunkRecord 引用
```

关键观察：

- **EP/DP Urma** (T9, T19) 只在 SU；与 SO 上的所有流量（KV bytes 数据面 + uRPC 控制面）物理隔离，**互不阻塞**。
- **SO 网络上同时承载** ① KV 字节 Urma read/write (T7, T12, T18, T21) 和 ② 控制面 uRPC (T2, T3, T5, T10, T11, T13, T15, T16, T24)；二者通过独立 Urma QP 调度，控制面短包不被大块 KV IO 阻塞。

- **F** 在 **T2、T5、T16、T22、T24** 共 5 次出场；调度器是**编排者**而非数据搬运者。
- **NPU↔SSU 的 SO Urma 流量** (T7, T12, T18, T21) 由 NPU 自驱，不经任何 CPU 转发；这是吃满 SO 带宽的前提。
- **DCN 在 T0 / T22 承担外部 user ↔ F 的 HTTP/gRPC ingress 与回流**；此外的运维 / 监控 / 部署等 DCN gRPC 调用与请求生命周期解耦，不在此时序图中（详见 §5.2.2）。

5.6 流量量级与 SU/SO 隔离论证

为说明"EP/DP 必须独占 SU"的合理性，把 C 档（128 NPU）decode 单 step 的各类流量按网络分桶（V4 Pro、batch=32 token、80 层、SU 单 NPU 3.2 Tbps、SO 单 NPU 400 Gbps）：

流量	网络	协议层	单 step 单 NPU 量级	单 step 总量 (128 NPU)	占用率（占该网络该端口）
EP all-to-all（FFN dispatch + combine, 每层 2 次）	SU	Urma	~150 KiB × 2 × 80 × 32 ≈ 750 MiB / step	~94 GiB / step	750 MiB / 100 ms（粗估 step）≈ 60 Gbps；占 SU 3.2 Tbps ~2 %
DP-attention 同步（barrier 小 payload, 每层）	SU	Urma	≪ 1 MiB / step	小到可忽略	< 0.1 %
KV 字节读（DN ← S, 每层 1 次）	SO	Urma read	V4 Pro 80 层 × ~5 KiB（混合层平均）/ token × 32 token ≈ 12 MiB / step	~1.5 GiB / step	12 MiB / 100 ms ≈ 1 Gbps；占 SO 400 Gbps ~0.25 %
KV 字节写（DN → S, 新 chunk, 512 token 一次）	SO	Urma write	每 ~16 step 一次 ~5 MiB / NPU	累计平均 ~0.3 MiB / step	< 0.1 %
PN → DN 末位移交（仅 prefill 切 decode 时）	SO	Urma	几 KiB	几 KiB	噪声级
控制面 RPC（PrefixMatch / AllocateChunk / CommitChunk / Dispatch）	SO	uRPC over Urma	~几 KiB / RPC × ≤10 RPC / step	≪ 100 MiB / step	< 0.05 % ，与数据面 Urma 通过独立 QP 隔离
外部 ingress（user 入站请求）	DCN	HTTP/gRPC over TCP/IP	仅 N_req × 几 KiB / s	不参与单 step	与系统内部完全解耦

单看占用率似乎都很轻，但隔离的核心理由是延迟而不是带宽：

- EP all-to-all 是 **强同步 barrier**：每层都要等所有 NPU 完成 dispatch 才能进入 MLP，再等 combine 完成才能进下一层。**任何抖动都直接乘以层数（80）累加到 token 级延迟。**
- 若让 KV 字节读/写也挤进 SU（即便带宽够），SO 上 Urma 大块顺序读会**周期性占用交换缓冲与端口**，挤压 EP all-to-all 的尾部小包，**p99 延迟显著放大。**
- v1 设计选择**物理隔离**：SU 上只跑 **EP/DP Urma**，SO 上跑 **KV 数据 Urma + 跨域 NPU↔NPU Urma + uRPC 控制面**。SU 与 SO 在硬件上是**不同网卡 / 不同交换平面**，从根上避免抖动。

SO 内部进一步隔离：数据面 Urma vs 控制面 uRPC：

虽然 KV 字节读 / 写与 uRPC 控制面共用 SO 物理网络，但两者带宽量级差 ≥ 2 个数量级（数据面 GiB/s vs 控制面 < 100 MiB/s）。Urma 的 QP 模型让二者：

- 用不同 RC QP 派发，独立 doorbell + 独立 completion queue
- 网卡 scheduler 按 QP 加权轮询，控制面短包永远不会被 64 KiB+ 的数据块阻塞 RTT
- uRPC RTT 目标 < 100 μs（即便 SO 上同时跑满 KV 读流量），与 §5.4.1 / §5.4.3 的关键路径承诺一致

5.7 与 §1.3 / §3.6.6 的交叉对齐

§1.3 网络分工	在 §5 矩阵中的体现
SU = 域内 NPU↔NPU 高带宽（Urma）	(PN, PN)、(DN, DN) 单元格的 SU Urma 标注；只承载 EP/DP
SO = 跨域 any-to-any（Urma 数据面 + uRPC 控制面）	数据面：(PN, S)、(DN, S)、(PN, DN)、(PN, PN 跨域)、(DN, DN 跨域) Urma；控制面：(F↔M)、(F↔PC/DC)、(PC/DC↔M) uRPC
DCN = 外部 ingress + 运维 / 监控 / POSIX FS（§1.7）	不进入 hot-path 7×7 矩阵；承担外部 HTTP/gRPC ingress、运维与管理面 gRPC、POSIX FS（详见 §1.3 / §5.2）
服务器内部 UB	(PC, PN)、(DC, DN)

§3.6.6 端到端图	在 §5 矩阵中的体现
① PrefixMatch RPC	(F → M) SO uRPC
② ChunkRecord list 返回	(M → F) SO uRPC
③ 调度注入 worker	(F → PC / F → DC) SO uRPC
④ host → NPU mailbox	(PC → PN / DC → DN) UB
⑤ NPU 直接发起 Urma read/write	(PN → S / DN → S) SO Urma
⑥ KV bytes 直接到 NPU HBM	(S → PN / S → DN) SO Urma
⑦ Commit 回环	(PC → M / DC → M) SO uRPC

5.8 Tokenizer / Detokenizer 部署位置

prefix cache 命中的判定基于 token id 序列（§3.5.4）；本节定义 tokenize / detokenize 这两个文本 ↔ token id 的边界函数在系统中由谁负责。

5.8.1 v1：tokenize 与 detokenize 都集中在 F（service_access）

阶段	谁做	在哪一步	节点
tokenize （plain text → u32 id 序列）	F	接收外部 HTTP/gRPC 请求后、调用 PrefixMatch 之前（§5.5 时序图 T1）	鲲鹏 service_access
detokenize （u32 id → plain text 片段）	F	DN 返回 token id 后、回流给 user 之前（§5.5 T22）	鲲鹏 service_access

为什么 tokenize 必须在 F

约束	推论
PrefixMatch 输入是 chunk_hashes（基于 token id），见 §3.5.4	F 调用 meta server 之前必须已 tokenize

约束	推论
外部 user 进来的就是 plain text, IP 边界在 F	F 是 token-id-vs-plain-text 转换的天然位点
单一事实源 (single source of truth)	整集群只有 F 这一处持有 tokenizer 配置; BPE merge 顺序、特殊 token 表、chat template 处理任何一处不一致 → chunk_hash 不一致 → cache 永远命中不了
Worker 抽象边界干净	PC / DC / PN / DN 完全不需要知道"文本"概念, 只处理 token id + KV + logits

为什么 detokenize 也放 F (v1 简化)

- 与 ingress 对称; 只有 F 持有 tokenizer 状态 → 集群中无 tokenizer 状态需要同步
- **stop sequence**、**banned words**、**sampling penalty**、**role-tag 检测**这些都是文本级策略, 集中在 F 一处实现自然
- DN→DC→F 的回流即便经过 SO uRPC 也只 < 100 μs, token id 单包仅几字节, bandwidth 不是瓶颈
- streaming 场景下 BPE 字节边界 (partial multi-byte token) 的 buffer 处理只需在 F 做

CPU 算力评估 (鲲鹏 service_access 单实例可承载)

指标	量级
tokenize 速率	鲲鹏多核 CPU 上现代 BPE / SentencePiece 实现 ≥ 几百万 token/s
单 8 K token prompt	< 几 ms
detokenize 速率	与 tokenize 相当; streaming 单 token 下 ~μs
相对 1 M context prefill 的 ~秒级	量级低于噪声, 不构成 TTFT 瓶颈

5.8.2 v1.5 / v2 可选优化: detokenize 下沉到 DC

streaming 输出对 **per-token 延迟** 敏感的场景下, 可把 detokenizer 下沉到 DC (decode host CPU), 把 token id → text 一步在 DC 完成、直接发回 F 的 token 流:

维度	v1 (F 集中)	v1.5 (DC detokenize)
单 token 回流路径	DN → DC → F (uRPC, id) → F.detokenize → user	DN → DC.detokenize → F (uRPC, text fragment) → user
单 token RTT	~150 μs (DN→DC UB + DC→F SO + DCN/TCP)	~150 μs (同上, detokenize 时间几乎一致)
stop sequence 早停 (关键收益)	F 才能判断; DC 必须先把 id 发到 F	DC 直接判断 , 命中时立即停止下个 step → 省 1-2 step 计算
tokenizer 状态分布	仅 F	F + 每 DC 节点 (必须严格同步配置)
升级 tokenizer 的复杂度	改 F 一处	滚动重启所有 DC + F

v1 不做的原因: v1 验证阶段更看重正确性而非 streaming 极限延迟; DC detokenize 引入"分布式 tokenizer 状态一致性"问题, v1 不值得。

5.8.3 多模型 / 多 tokenizer 的统一处理

情况	处理
V4 Pro 与 V4 Flash 同 tokenizer (DeepSeek 同家族)	F 上加载一份 tokenizer 即可服务两个模型; 但 cache key 要拼 model_id (§3.6.2、§3.5.4) 防错命中

情况	处理
不同 tokenizer 的模型并存	F 上加载多份 tokenizer，按请求中的 model 字段路由
tokenizer 升级	model_id 必须递进，旧 cache 自动作废（不会假命中）

5.8.4 Config 钩子

```
service_access:
  tokenize:
    placement:          frontend          # v1: 仅在 F；v1.5 备选 frontend_and_decode
    tokenizer_configs:  # 一份模型一项；按 request 的 model 字段路由
      - model_id:      deepseek-v4-pro-r1
        tokenizer:      deepseek-v4-tokenizer
        chat_template:  deepseek_v4_chat
      - model_id:      deepseek-v4-flash-r1
        tokenizer:      deepseek-v4-tokenizer # 与 Pro 同一份；但 model_id 仍隔离 cache
        chat_template:  deepseek_v4_chat
  detokenize:
    placement:          frontend          # v1: 仅在 F；v1.5 备选 decode_worker
    streaming_buffer:    bpe_partial_token # 处理多字节 token 的字节边界
    stop_sequence_check: at_frontend      # v1.5 切到 at_decode_worker 可早停 1-2 step
```

5.8.5 与 §5.5 时序图的精确定位

补充说明 §5.5 时序图中两处文本转换：

- **T1**： F.tokenize(prompt_text) → token_seq[N] —— 在 F 完成；后续所有节点对内只见 token id
- **T22**： F.detokenize(token_id_n) → text_fragment —— 每生成一个 token 经 DN→DC→F (SO uRPC) 回流到 F，F 负责文本拼装与 stop sequence 检测，再走 DCN/TCP 发给 user

整条请求路径上文本只在 **user ↔ F** 之间存在，**集群内部只流通 token id**。

5.8.6 与 §4.7 的边界 ("text↔id 在 CPU、id↔vector 在 NPU"两清)

§5.8 与 §4.7 处理的是**两层完全不同的"编码 / 解码"**，部署位置与实现机制都正交：

层	输入	输出	实现	部署	覆盖章节
第一层 (CPU) ： tokenizer / detokenizer	plain text	u32 token id	BPE / SentencePiece, CPU 算子	F 鲲鹏 CPU	§5.8
第二层 (NPU) ： vocabulary encoder / decoder (embedding / LM head)	u32 token id	bf16 hidden / logits	gather / matmul, NPU 算子	PN/DN HBM	§4.7

两清边界：

- **F 永远不知道 hidden vector / logits** (NPU 内部细节)
- **NPU 永远不知道 plain text** (仅 token id)
- 这套两层分离让运行时配置 (tokenizer 升级 / 模型替换 / 采样参数变更) 完全解耦：tokenizer 升级只动 F；模型升级只动 PN/DN；sampling 参数 per-request 透传不影响任何全局状态。